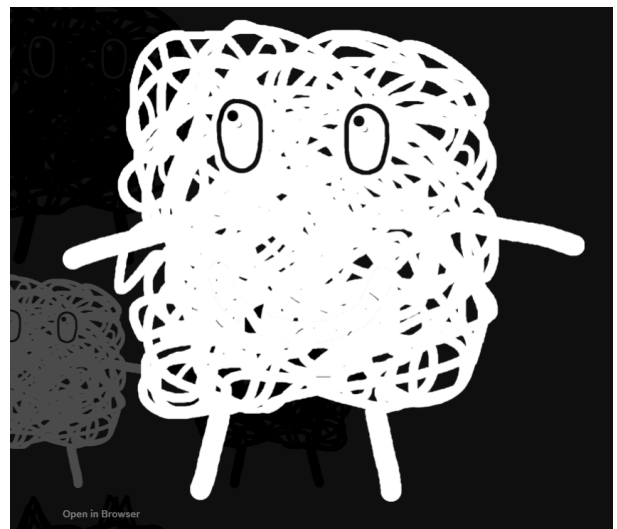
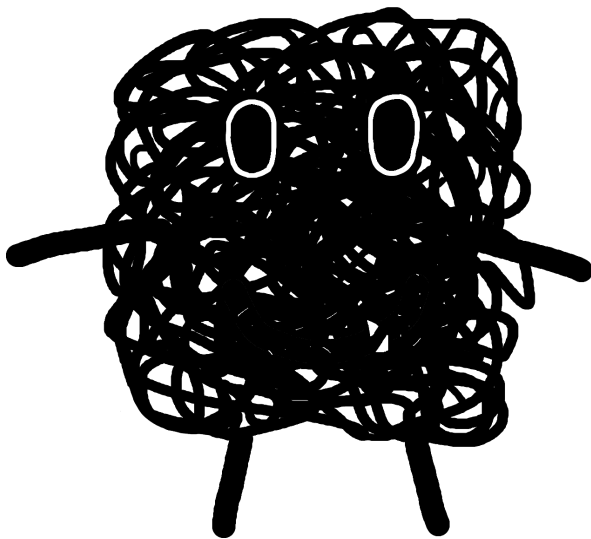


Scrubble

Anti-Malware and Network Protection Service

An Investigation into effective forms of malware identification



By Rajan Summan

Contents

Contents.....	2
Project Background.....	6
Investigation outline.....	6
Client - Interview.....	7
Research.....	11
What is an Antivirus and How does it work ?.....	11
Different kinds of malware and grayware.....	12
Social Engineering.....	15
Steganography.....	16
Malware prevention and Protection methods.....	16
Packet sniffing.....	17
Firewall.....	17
Encryption.....	17
DES and Triple DES.....	19
AES.....	20
RSA.....	20
Hashing.....	20
Ports target by hackers.....	21
Virtual Machines.....	22
Project Details.....	24
Scanning methods.....	24
Different checking methods.....	24
Simplicity / Hands off approach to security.....	25
Sandbox testing system.....	26
Extensive Database.....	26
User Interface.....	26
Potential Algorithmic Solutions.....	27
Local virus detection and extermination.....	27
Tree Traversal Algorithms.....	29

Mascot - Prototype design 1.....	30
Potential Programming Solutions.....	31
Python.....	31
C / C++.....	31
Assembly.....	32
Java.....	32
MySQL.....	32
PostgreSQL.....	32
MongoDB.....	33
HTML.....	33
Flask.....	33
Programming Solutions Chosen.....	33
Python.....	33
MySQL.....	34
HTML.....	35
Open Source useful tools.....	35
Zeek.....	35
ClamAV.....	35
YARA.....	36
Watchdog.....	39
Virtual Box.....	39
Cuckoo Sandbox.....	40
TkInter.....	40
PSUTIL.....	40
Github pages.....	40
Specific Investigation Objectives.....	41
Technical Skills Required.....	45
Design.....	46
System Overview.....	46
Key data structures.....	46
Hash Tables and Hash Functions.....	47
Trees.....	47
Arrays.....	47
Dictionaries.....	47

Key Algorithms.....	48
Graph and Tree traversal algorithms.....	48
Recursion.....	48
Brute force algorithm.....	48
Re-Ordering algorithm.....	49
IPSO Graphs (Input, Process, Storage, Output).....	50
Hierarchy Charts.....	50
Flow Diagrams.....	50
User Interface.....	51
Permanent data storage format.....	53
Database.....	53
Hash Table.....	54
Log files.....	55
Database Design.....	55
Prototype 1.....	56
Prototype 2.....	56
Configuration File.....	56
Internal Data Storage.....	56
Object Oriented approach.....	56
Tree.....	57
Priority Queue.....	58
Stack.....	58
Algorithms.....	59
Breadth first traversal.....	59
Heuristic Algorithms.....	59
Pseudocode.....	60
Sandboxing.....	61
Virtual Machine Setup and Configuration.....	61
Baseline Activity for System Processes.....	63
Technical Solution.....	64
GUI.py.....	65
Addition of different users.....	65
YARA Comparison.py.....	66
Compile_yara_rules().....	66
Check_yara_rule.....	67

Imbedded SQL - Threat table.....	69
Imbedded SQL - Scan table.....	71
Re-Ordering Algorithm.....	72
GUI_to_Yara.py.....	73
Watch Directory.py.....	74
Watch_Directory().....	74
Full Code.....	75
Quarantine Module.py.....	76
Hashing Algorithm.....	77
Virtual Machine.py.....	78
Process Analysis.py.....	78
Full Code.....	78
Index.html.....	78
Config.cfg.....	78
SQL and Database Management.....	79
Creating databases.....	79
Relating Databases.....	81
SQL Queries.....	81
Virtual Machine.....	95
Installing Virtual Box.....	95
Configuring Virtual Box.....	96
Virtual box Automated code.....	97
Testing.....	99
Test Plan.....	99
Conclusions of the Investigation.....	104
References.....	104
Glossary.....	106
Full Code Appendix.....	108
GUI.py.....	108
Yara_Comparison.py.....	112
Index.html.....	119
Config.cfg.....	124

Project Background

Ryan Montgomery is an ethical hacker and **penetration tester**. He has been popularised on social media platforms such as tik tok and instagram for his work catching child predators and sexual assaulters using his hacking skills. He has been a great inspiration towards me personally and has been a core reason for my interest in cyber security and **digital forensics**.

Technology is an ever increasing part of our lives, slowly taking over and optimising everything around us and becoming more and more sophisticated by the minute. Therefore, the more technology is used, the more personal data it can gather about users (such as bank statements and home addresses.) Therefore people such as Ryan who have a large sphere of influence preach the importance of digital defence systems, as malware and other forms of cyber attacks are evolving at the same rate as the rest of the technological sector.

Unfortunately, large cyber security companies gatekeep the security programs like antiviruses that could protect so many vulnerable users from online attacks behind large paywalls and subscription services.

I would like to help this issue by investigating how anti-malware software works to help educate and protect unsuspecting users about online threats without having to pay large amounts of money.

Therefore for my project, I will be investigating effective methods of malware identification methods. By researching identification and extermination methods, I aim to determine which malware detection methods are the best, and how we can optimise these methods to make them the most efficient.

Investigation outline

I am doing an investigation into malware identification. In order to achieve a thorough investigation into this, I will create a series of tools that will allow me to demonstrate the various methods of identifying malware, and test them in a variety of different scenarios to help me identify the strengths and weaknesses of each method

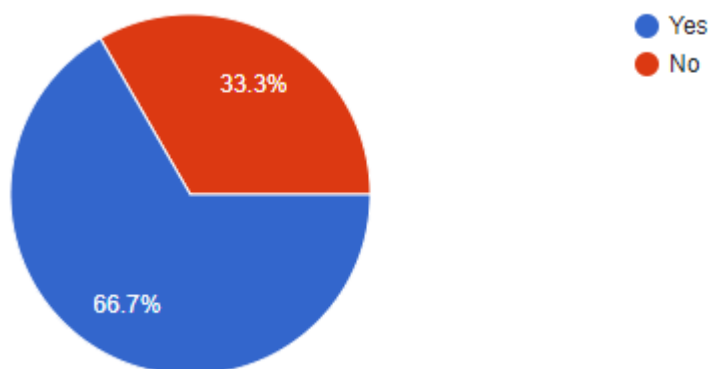
Current users - Interview

Before conducting the investigation myself, it is important to gain an understanding of how much the population values their cyber security. As we approach an ever increasing digital world, there is an increasing need to keep people safe online.

To do this I asked a variety of questions regarding the antivirus they use if they use one at all. This helped me gain an insight into what factors of an antivirus persuade people to choose to use that particular company to protect themselves and their data.

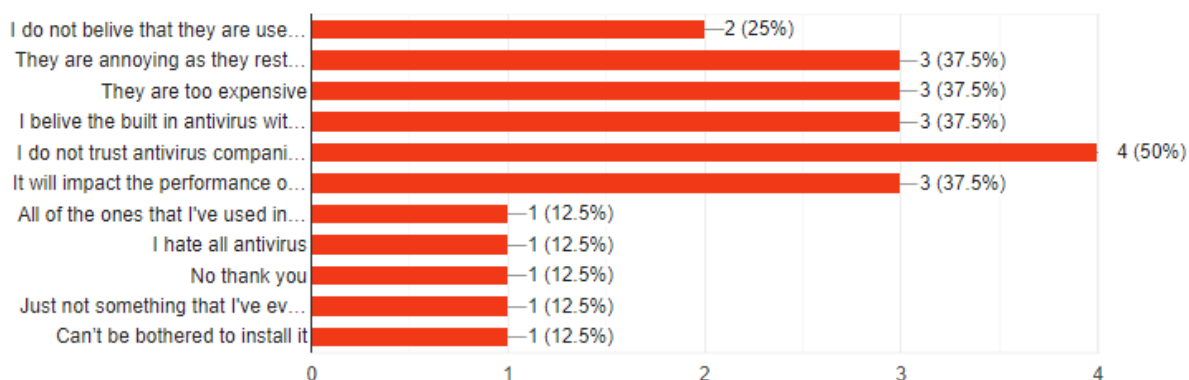
To conduct market research, I created an online survey and handed it out to various people online and got 24 responses. It ranges from people with a deep knowledge of computer science and the implications of pure cyber security to people who have very limited computing knowledge.

This is the response from the survey where participants were asked if they had any form of anti-malware software installed on their personal computer



Although the majority of users ($\frac{2}{3}$'s) of the audience did have the antivirus, a shocking $\frac{1}{3}$ admitted to not having any form of antivirus software installed. However It is worth mentioning that potentially users do not know that their operating system has antivirus software built in, therefore to improve this question in the future, I would clarify this with the participants, and then ask them to see if they chose to willfully ignore it. Rather than clarifying in question 2 that their operating system had an antivirus.

For users who selected no to why they had not installed an antivirus beyond, they were asked to select for a series of statements to best suit their personal reasoning for why they decided to not improve their online safety further. The results of this are shown below.

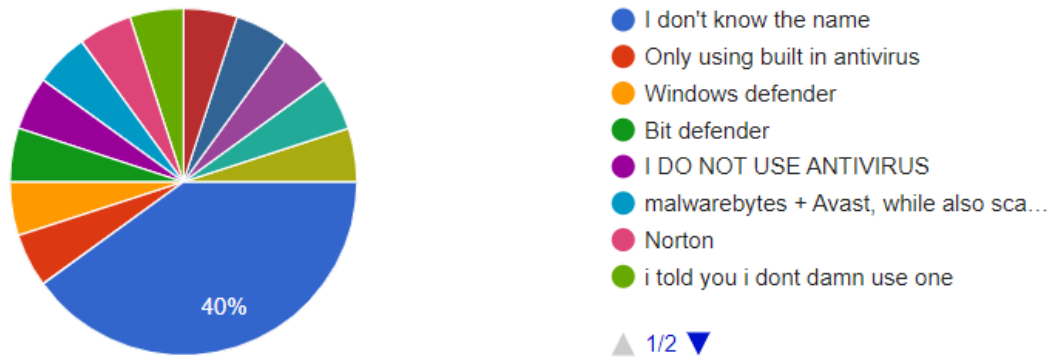


As shown below, half of the people who stated that they did not have an outside antivirus installed said that It was because they did not trust the large data companies, this was the majority. This clearly shows that my participants are very aware about the threats online, however it is worth noting that my participants are made up entirely of people between the

ages of 16 and 21, which are typically much more aware about online security. Typically online attacks target much older and younger people who are less experienced in using computer systems and therefore have their naivety exploited. Therefore to improve this research, I would try to gather a wider audience of participants of various different demographics to attempt to eliminate the bias seen in this data.

Secondly, a large proportion of people who don't have antivirus software state that it is because they are annoying to use because they restrict online access. Personally, I reside with this point as well. Therefore to cater to the needs of the younger generation where they would like to have free access while still being protected, My antivirus will not be able to block direct access to sources online, It will warn users and suggest leaving the site if potentially suspicious activity is detected, however it will not restrict anything all together. I also believe that if I can refine the local file quarantine and extermination process enough, navigating malicious areas should be able to be countered locally rather than online.

Additionally, It is worth mentioning that users are also concerned about how the antivirus software will impact the overall performance of the users computer system. This is because current systems on the market will run constantly therefore allowing an extra task that the PC has to multitask which for users who have weaker older generations machines, this can lead to major performance drops resulting in the users uninstalling there antivirus altogether and leading to users compromising their online safety for efficiency. To help fix this issue, my product will only scan the user's drive when they feel like they need to and lie dormant when it is not in use. Therefore performance will not be impacted as much as much, hopefully resulting in the users making frequent checks to the health of their storage drive.



For users that did have an antivirus installed on their local machine, by far the most common response is that they did not know the name of their antivirus. The most logical conclusion as to why this is is because users simply don't care much about which kind of anti-malware software. Therefore I believe that it is very important to educate users about how important it is to keep their data safe.

It is also worth noting that this point is further enforced by the second question I asked the participants. This was to gather information as to why the user decided to install the antivirus software they chose over other alternatives in the market.



As shown in the pie chart above, The majority of users said that they chose their antivirus software because it was recommended by family or friends, which again could leave these users vulnerable to phishing attacks if the scammer was able to pose as a friend or family member.

To conclude all of the information I conducted from this survey. I found that the general sample are very unaware about their antivirus software.

This shows that the sample does not consider their own cyber security and simply puts their trust into their downloaded antivirus.

Additionally, it shows how much people value efficiency and optimisation, even if it does sacrifice security slightly. Therefore I believe my investigation into the most efficient and thorough methods of malware identification is applicable, since consumers will prefer a faster antivirus.

Finally, people also have issues trusting the antivirus companies that provide the service, which is understandable since they essentially have access to all of your files locally and online. However, I believe that informing people on how antivirus software works is crucial to regaining this trust to try and protect more people from cyber crime. This is why I want to drill down into this investigation to gain a deep understanding of how antivirus software works, so I can share this information to hopefully rebuild a level of trust within the user base so they will trust their antivirus to keep them safer online.

Research

What is an Antivirus and How does it work ?

In short, an Antivirus is a form of software that will scan files in a computer's memory looking for certain patterns of binary that may hold the code for malicious software. It does this by referring to a large database holding all traces of known malware, then using this to identify the malware. This is known as a **dictionary approach**

However, an obvious flaw of this approach to defending against threats is that if a new threat that is not in the database, It will not be picked up

by the software and therefore will reach the user and perform its malicious attack. Therefore to counter this, software will take a **heuristic approach**. What this means is that the software will look and monitor the behaviour of software and look for suspicious activity.

The impact of this is that users have an additional layer of security and assurance that their computer will be able to detect any form of malicious activity and will provide full protection of their data.

Therefore for my investigation, I will create a tool that can scan a file system using a dictionary approach and create a different tool that can use a heuristic approach to identify malware. Then I will test both and compare how efficient and effective each was at identifying the malware.

Different kinds of malware and grayware

Widely speaking, software in general can be separated into 3 different categories:

- **Goodware** - Software that is downloaded from a trusted source
- **Greyware** - Software that is not from a trusted source, but isn't directly harmful to the device or user
- **Malware** - A form of software that is designed to harm a computer system

My program will need to be able to tell the difference between these 3 types of software and specifically hone in on malware.

During my studies of computer science at both GCSE and A-level I already have some prior knowledge on: Viruses, worms, trojans, botnets, ransomware, spyware, adware. However upon further research, there appears to be upwards of 40. My software will have to be able to defend against these different kinds of malware.

Virus Name	Description	Example
Adware	Software that is designed to automatically display advertisements to the user, usually to collect revenue. It most commonly takes the form of pop up ads or pop up videos. Is a form of grayware	Gator - popular in the early 2000's
Air - Gap malware	An advanced form of malware that is designed to target systems that are disconnected from any forms of network. It targets peripheral devices like microphones or USB drives to connect and extrapolate data.	Stuxnet - found in the 2010s
Automated threat	A type of threat that uses bots to attack a user. They typically replicate a large amount of human activity	Spambots
Blended threat	A combination of different forms of malware to find an exploit within a system.	Paris attacks - 2015 where partially cyber and physical
Browser Hijacking	A form of software that will change the users settings of their web browser without the user knowing. Typically this is used in combination with adware or spyware. This is a form of greyware	Astromenda
Code injection	When an attacker will introduce code into a computer program to change the way it is executed. These are most common in SQL to extract data that is supposed to be hidden.	SQL injection
Computer Virus	Software that upon execution, will replicate itself by infecting other neighbouring programs. They require a host program to execute. A virus is typically composed of 3 major components: the infection mechanism, payload and the trigger. The mechanism or infection vector - is how the virus spreads The trigger or logic bomb - is what activates when the host program is ran The payload - is the bulk of the virus that executes the malicious activity. Similarly to human devices, computer viruses have a life cycle that consists of phases. These are: Dormant phase, Propagation phase, triggering phase and the execution phase. Dormant phase - The program is idle and not currently running Propagation phase - the program begins to multiply and replicate once the trigger is activated. Triggering phase - The virus has first been activated Execution phase - When the payload is released and destruction occurs	Brain - discovered in 1986.
Computer Worm	A malicious program that is similar to a virus will replicate itself to spread and attack other files or other computers on a network. However the primary difference between a worm and a virus is that a worm does not require a host program. This is because it is independent from any other file and therefore can be run independently.	I love you MyDoom
Crimeware	A form of subclass of spyware and adware that is designed for the use of	MPack

	identity theft in order to gain access to users personal assets. Such as online banking.	
Trojan horse	A computer program that masquerades itself as another standard program to trick the user into installing and running it. It is named after the story of the Trojan horse in greek mythology.	I love you - was initially downloaded using an email
Fleeceware	A form of malware typically found within mobile applications or games that have a large amount of subscription fees blocking its use. It takes advantage of younger people who sign up for the subscription without properly reading what it is or how to cancel it. This is a form of greyware	A flashlight app charging 8 pounds a month !!!
Form grabbing	Form grabbing is when a fake login page tricks the user into giving over usernames and passwords, or other forms of credentials, before relaying them back to a form of server, which can then be exploited to gain access to personal information.	Tinba - was able to steal online bank details
Malvertising	The usage of advertising online to trick users into downloading malicious software.	
Ransomware	Once installed, the program will block off the users access to their own personal data behind a paywall and will promise to give the data back once the ransom has been paid. It typically encrypts the data and then charges for the decryption key. However there are other types of ransomware that don't use encryption	Cryptolocker
Leakware / Doxware	Another form of cryptovirology attack that threatens to publicly reveal stolen private information unless the victim pays a fee.	darkside
Rootkit	A rootkit is a bundle of software that is designed to allow unauthorised users access to a computer or network, this is all without being detected.	Sony BMG copy protection
Scareware	A cyberattack that scares people into using tapped websites usually in the form of pop up ads saying how their computer has been infected.	Mac Defender
Spyware	Software that will maliciously track the users behaviour to gather information. Examples of this include key and mouse loggers to extract details when users are entering home addresses or bank statements into websites, or using webcams and microphones to spy on the users actual location.	Keyloggers
Wiper	A type of malware designed specifically to erase the contents of the victims local storage.	Shamoon
Zip bomb	Also known as a decompression bomb, is a simple type of malware that will crash a computer system when unzipped. It is done by compressing large amounts of data within a zip file so that when the file is unzipped it will crash the computer	Email bomb

Social Engineering

Social engineering is the term used for the wide range of malicious activities accomplished through human interactions. By using manipulation tactics, it will lead to the attackers gaining access to sensitive information giving further access to systems that they otherwise would not have had. These are tricky to intercept using software as they are done by real people in the real world rather than digitally. Therefore social engineering is incredibly effective as in the modern world, the most complicated defence systems can be bypassed by exploiting human trust. Social engineering comes in many forms. The only way to counter or defend against social engineering is to educate users on its threat and encourage them to take action against it themselves, such as making the passwords harder to guess. I would like to add an information feature within the project to inform users on social engineering.

Types of Social engineering

Name	Definition	Example
Shoulder surfing	Shoulder surfing is the process of visually looking at you input sensitive data into a website by looking over your shoulder at the device	Watching someone input their pin at an ATM
Phishing	Phishing is the process of an attacker using scam emails, texts or phone calls to trick users into giving over sensitive information or leading them to a malicious online website	Scam insurance calls and emails.
Whaling / spear phishing	A form of phishing attack that is specifically targeted towards a specific person to make it more convincing	A threat to a CEO making them wire transfer funds
Baiting	Baiting is when an attacker uses a false promise to lure victims into a trap to extract sensitive information.	Nigerian prince scams
Diversion Theft	Diversion theft is when an attacker will divert attention away from another area they wish to exploit.	DDOS attacking a network to extract info from a server
Quid pro quo	Quid pro quo stands for "something for something." Therefore quid pro quo is where an attacker will pose as someone who they are not to extrapolate sensitive information	A fake tech support worker.

Pretexting	When an attacker uses a fake story to exploit a victims trust and tricking them into giving over personal information.	Someone impersonating a supplier for a businesses
------------	--	---

Steganography

Steganography is when someone will conceal and hide information or a message within another object to evade detection. This can be applied in both a physical and digital context. It has close relations to and is often used with encryption.

Steganography is therefore something we have to consider since files that we are scanning might at first glance appear to be one thing, but there are actually hidden properties of that file that may or may not be malicious. Trojans are a good practical example of steganography in process.

Malware prevention and Protection methods

Antiviruses are one way of preventing malware from impacting a computer, however there are multiple other ways of protecting a computer against harmful outside threats. It is important for me to be aware of these methods to ensure my code is best optimised to work alongside these other security measures.

I aimed to create a project capable of intercepting harmful packets from a network in real-time to prevent malware from infecting users' local storage drives. This required a deep understanding of network threats and the communication between users and the online sources of these threats.

Packet sniffing

Packet sniffing, also known as packet analysing, is a form of computer program that detects and assesses the data within each packet being sent over a network. However, due to the Data protection act of 2018 this is made illegal to apply to the wider internet, due to the packets being searched may contain sensitive information of the users.

Therefore in order to implement this feature, I would have to use either a network owned entirely by myself or using a virtual network and machine. I can use the Cisco packet tracer to help model the network and packet sniffing feature as a method of protecting networks

Alternatively, rather than viewing the contents of each packet, I can use a separate approach that instead focuses on how many packets are entering and leaving a network in an amount of time. This allows us to gain an understanding of the communication on the network.

Firewall

A firewall is another method of intercepting potential malicious packets. It works by in effect configuring a proxy or router to accept or reject each packet that is being received from the network based on a set of rules or configurations made prior. A firewall can be hardware or software based. This may potentially be a good form of network security to help prevent malicious packets from reaching the user's PC.

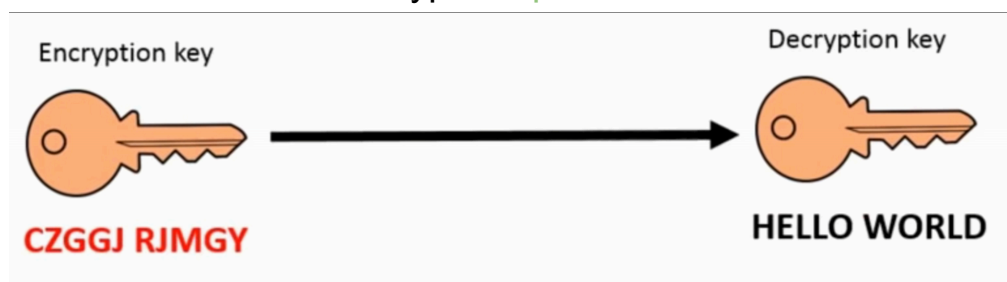
Encryption

Encryption is a term that has existed long before computer science was created as a field of study, it was first used in 1900 BC being the use of the caesar cipher which is still a popular choice of encryption today. Within the study of cryptography, encryption is the process of using an

algorithm to scrambling data or “plaintext” to make it unreadable for both humans and computers without the proper algorithm to descramble the message or data. These algorithms are known as Keys. Encryption is a wide topic with many different ciphers that all function in many different ways. While it is technically possible to decrypt a message without the paired decryption key, it requires a large amount of willpower and time of guessing combinations, upwards of years or even centuries making it a very secure method of network security. It's brilliant at stopping potential malicious hackers from gaining easy access to sensitive information from a network.

From my studies of A-level computer science, I have learned that there are two types of encryption keys that cyber security experts use

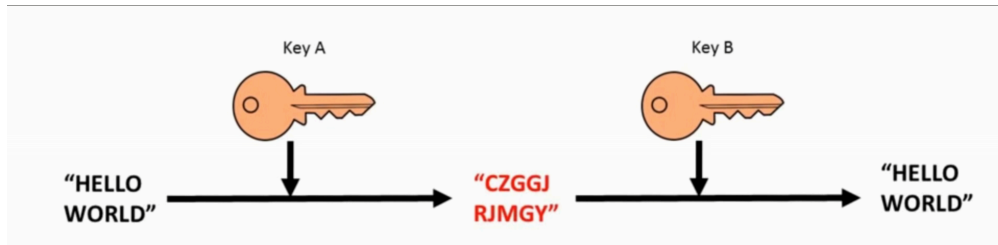
- Symmetric - involves using the same key to encrypt and decrypt the message. This form of encryption is very simple to understand and apply, however is less secure as only one key needs to be obtained in order to decrypt the plaintext



- A potential workaround to the security issues created by symmetric encryption is using a technique known as a one-time pad. This uses a fully randomised private key that can only be used once to encrypt a message. This is then decrypted by the receiver using a corresponding one-time pad and key.

By using this method, the code becomes unbreakable with no true method of interception, however this still arises the same issue as before that if the attacker is able to access the key in the one time pad, the message can be read easily.

- Asymmetric - involves encrypting the plaintext using one key, and then decrypting the key using another key. It is critical that the two keys are different from each other for this form of encryption to work effectively.



These keys can then be applied in various different ways to create different encryption algorithms or ciphers, the most common ones used in industry today are:

- Triple DES
- AES (advanced encryption standard)
- RSA

Source - <https://www.indeed.com/career-advice/career-development/types-of-encryption>

DES and Triple DES

DES (Data encryption standard) was developed in 1971 by IBM and has been used by agencies as large as the US government in the 70's. It uses a symmetric encryption key that encrypts a mere 56 bits making it susceptible to brute force attacks from malicious outsiders, This was made worse by the single key being only 64 bits. This means that there were only 18,446,744,073,709,551,616 keys in existence. Therefore it was upgrade into Triple DES

Triple DES uses the same symmetric encryption method, but reinforces the key by a factor of 3. This means that the algorithm is applied 3 times and it can now encrypt up to 168 bits of data, however the 56-bit encryption key remains making it still very susceptible to attacks, therefore regarded as widely impractical although it is simple to implement.

AES

AES or advanced encryption standard was made to be a replacement for DES as a standard, introduced in 2001. AES can be used for 128, 192 and 256 bits of data. It operates using a substitution-permutation network or SPN. What this means is that blocks of plaintext and keys are applied in alternating rounds of substitution layers and permutation layers to produce the final encrypted message. This makes it much more secure and harder to crack than DES

RSA

RSA is a form of cipher that uses asymmetric encryption. It generates a **key pair** based on the product of two large prime numbers. Then uses specialised mathematical functions to differentiate between the public and private keys. To encrypt the message, the recipient's public key is applied to the plaintext before being sent, once received, the recipient's private key is applied to decrypt the message.

Hashing

Hashing is the process of transforming input data into another unique format to produce a unique fingerprint for that file. This fingerprint is known as a hash value, and the algorithm used to create the fingerprint from the input data is known as a hashing algorithm.

Cyber Security specialists will use hashing on the identified malicious files for multiple reasons:

- By applying a hash value, it ensures that there is no corruption or activity in the file, since if there was there would be a change in the raw data leading to the hash values not matching. This insures the files integrity. It's also useful for ensuring that the correct file is the one being deleted

- Its useful for keeping a record of the files identified, which allows use to trace these files back to their source before any manipulation took place which is useful for cyber forensics

Potential Hashing Algorithms

SHA-256 - The most common hashing algorithm that produces a very secure 256-bit hash

SHA-1 - an insecure 160-bit hash

Ports target by hackers

Port 445

Port 445 is known as the Server Message Block Protocol or SMB protocol. It's responsible for sharing files within windows networks. It's a relatively open port meaning that hackers will often exploit weaknesses within this protocol for both ransomware and worm attacks.

Port 23

Port 23 is the default port for telnet. It allows for remote connection to other devices. While outdated and generally replaced by port 22 it is sometimes still used. However it has little to no authentication within it and the data is not encrypted making it a target by hackers for man-in-the-middle attacks.

Port 22

Port 22 also known as the Secure Shell Protocol (SSH). It's used for remote access to devices where it's encrypted, making it superior to port 23. It's often a target of brute force attacks when the user login details are weak.

Port 80 & Port 443

Port 80 & 443 are used for communication with web servers. This makes them a target for exploitation that are web-based, such as SQL injection.

Port 3389

Port 3389 is known as the remote desktop protocol similar to port 445 is used for remote access to windows machines. It's prone to attacks to try and gain access to the remote machine via brute force.

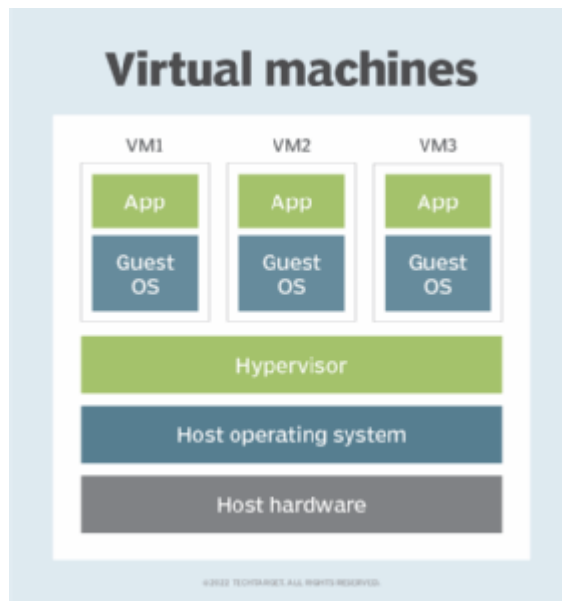
Port 53

Port 53 is responsible for running DNS (Domain name servers). It translates the domain name entered by users in a browser into the IP address of the web server. It's sometimes targeted by hackers by using fake dns'

Virtual Machines

Virtual machines are essentially simulating or emulating a computer using software using the physical resources or hardware of another computer. It functions no differently than a physical computer like a laptop, except all of the hardware is emulated through software.

Virtual machines are invaluable to cyber security experts as they allow us to run tests and study apps in a secure environment. This allows us to verify that the application is actually dangerous without placing the host computer at any sort of risk. I would like to implement a virtual machine into my project to



This diagram demonstrates how the virtual machine sits on top of the host hardware and operating system which is what protects it.

However, I will need to upgrade my operating system on my local machine from windows 11 home edition to windows 11 pro edition, which can only be done using an activation key given by Microsoft themselves. This is needed to enable Hyper-v which allows for hardware virtualisation.

Additionally, an ISO file of the operating system you intend to run on the VM is required.

Fortunately, My dad who owns an IT company that has licensing with Microsoft, and therefore getting a copy of windows 11 pro was easy.

However, It is worth mentioning that testing malware within a virtual machine does inherently come with some levels of risks as human beings are prone to mistakes, and failing to configure the VM correctly can result in the malware jumping from the VM to the host machine. For this reason, I will not be testing any actual malicious software just to be safe. Additionally, another reason why I will not be using actual malware is that modern malware has anti-analysis capabilities in mind.

Project Details

Scanning methods

As mentioned previously, Antiviruses work by scanning memory on the user's computer and referencing a database. However If the user has terabytes worth of data worth of applications and files, searching each individual file will take a very long time, therefore I want to be able to provide the user the option to do a "quick scan" or an "in-depth scan". This is a feature already in multiple antivirus programs on the market and I would like to implement it into my own. In order to achieve this I must:

- Program 2 separate searching algorithms to traverse the users storage drive
- One must prioritise efficiency only checking for suspicious files in suspicious areas such as looking at specific folders such as downloads where trojans are likely to be located. And another that will do a more thorough search looking at every file and checking both against the database and checking its heuristic characteristics to identify harmful threats
- Provide the user upon startup of the application a choice as to how they want to check their device
- Give the user the freedom to scan a specific folder if they feel as though it may have a virus trace in it that the antivirus has missed.

Different checking methods

As I hinted at earlier, Antiviruses in the modern day typically use 2 ways of assessing whether or not a file is deemed as malicious or not. These

two ways are known as a dictionary check and a heuristic check. For my project, I would like to implement both of these into my project.

- Dictionary check: referring to a database filled with large amounts of malicious code traces, then checking the file to see if that code is contained within it
- To implement this, I will need to be able to query a database to pull the correct virus trace when a threat is found, then display to the user what the threat is and the best possible option to defend against it.
- Behaviour based Approach: Looking at the behaviour of software to detect suspicious behaviour in real time
- This helps to protect against polymorphic viruses (viruses that change and adapt), and can also help to detect viruses that don't have recognisable traces in the database
- To implement this, My program must be able to create a specialised virtual containment unit where the malicious code can be tested without the risk of the code spreading to other areas in memory, before looking for suspicious characteristics, such as self-duplication or overwriting files.
- Heuristic Approach: Uses algorithms to identify potentially malicious properties based on other existing viruses properties and characteristics.
- This can be implemented using a combination of both behavioural and dictionary analysis.

Simplicity / Hands off approach to security

- As gathered from my research survey, my target demographic finds that antivirus software is currently unintuitive. They are hard to use and are generally not pleasant to interact with and can sometimes be complicated to use. Therefore I want to use an eye-catching simple user interface.
- What I mean by a hands off approach to security is that currently, installing antivirus software will limit your online access to web

pages that users want to access even if they know it is risky, but they choose to accept the risk, therefore I want my system to acknowledge the threat and warn the user, potentially running an algorithm to calculate the threat level and displaying it to the user, but not entirely blocking or restricting access. This was also one of the primary reasons for users uninstalling their antivirus software as gathered from my survey.

Sandbox testing system

- Sandbox testing which I referred to earlier, is when an antivirus system will create a safe virtual environment where the malicious file can be tested to be certain that it is threatening. It does this by checking for properties found in malicious software, such as self duplication or accessing other files. It then will safely destroy every instance of the malicious software within that virtual environment.
- This can be achieved using a **virtual machine** such as virtual box

Extensive Database

- To be as reliable as possible, I want my database to be as current and up to date as possible with lots of different traces present. I can achieve this by using SQL to query lots of databases found online to gain as much information on their traces as possible before storing them all together

User Interface

- Something that was highlighted to me through my research is how important it is to have a simple user interface. This is because users care much about the aesthetics of the applications they use, this was highlighted to me during interviews I conducted by myself with a few of my classmates. Therefore I want to ensure that my

product is easily accessible to all people with easy set up and start up.

Potential Algorithmic Solutions

Local virus detection and extermination

Idea 1

For my project, I can program my software to iterate through each file within the user's storage drive, and then scan each file to ensure there is no malicious code within it. This can be done using iteration or **recursion**.

Prototype - 1

```
Python
import os

directory = input("Please enter the directory you wish to scan :")
os.chdir(directory)

cwd = os.getcwd()

print("The current working directory is " + cwd )

print(os.listdir())
```

This very basic piece of code in its current state will take an input for a file directory, and display every file or folder within it. This basic skeleton can be the fundamental logic of the iterative approach, using this printed

list, we can now use a linear search to check each file and folder in the users directory and scan for virus traces.

File/Folder Function - Prototype 1

```
Python
import os

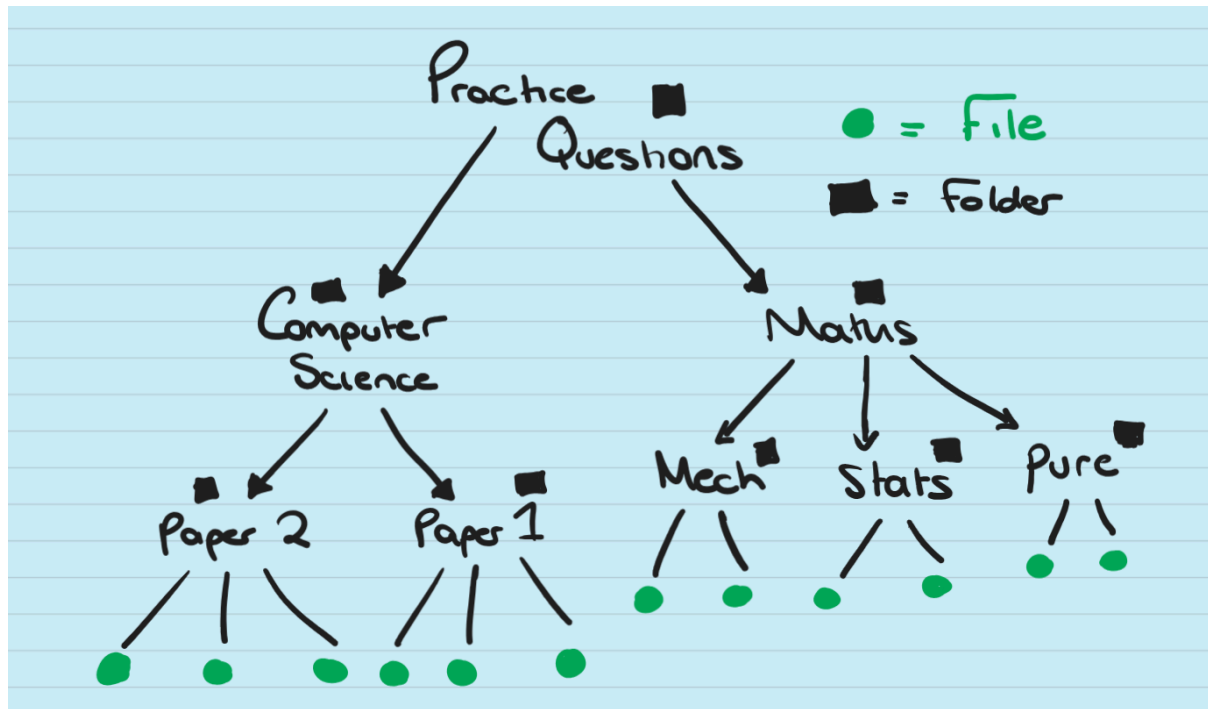
def check(file_list):
    for element in file_list:
        if os.path.isfile(element) == True:
            print(element)
        elif os.path.isdir(element):
            element = os.path.realpath(element)
            print("Files in directory:", element)
            for file_name in os.listdir(element):
                print( file_name)

directory = input("Please enter the directory you wish to scan :")
os.chdir(directory)

cwd = os.getcwd()
print("The current working directory is " + cwd)
file_list = os.listdir()
check(file_list)
```

By implementing this File/Folder function, I was able to print each item in the directory selected by the user. It searches the folder linearly and prints the item if it is a file, and opens and prints the contents of the folder, if a folder is encountered.

Here is a model of my personal folder for storing practice questions as a tree. Taking this is an example for an input by the user. I want my program to be able to scan every folder to reach the files at the bottom of the tree, however in its current state, it will only traverse one layer deep. As shown in the output terminal below.



```

Please enter the directory you wish to scan :C:\Users\rajan\Desktop\Practice Questions
The current working directory is C:\Users\rajan\Desktop\Practice Questions
Files in directory: C:\Users\rajan\Desktop\Practice Questions\Computer Science
Paper 1
Paper 2

Files in directory: C:\Users\rajan\Desktop\Practice Questions\Maths
Mech
Pure
Stats
  
```

Tree Traversal Algorithms

A computer storage drive can be modelled as a tree, as demonstrated above. Implementing a tree traversal method for the user's storage drive is a potential approach at identifying viruses, taking folders as **nodes** and files as **terminal nodes**. When terminal nodes are reached, the file will be scanned for traces of viruses.

Breadth-first traversal

The breadth-first traversal approach to traversing a tree works by exploring all the nodes in a graph at the current depth before moving on to the next level. It starts from any node, (in this case it will be the very top of the tree) .

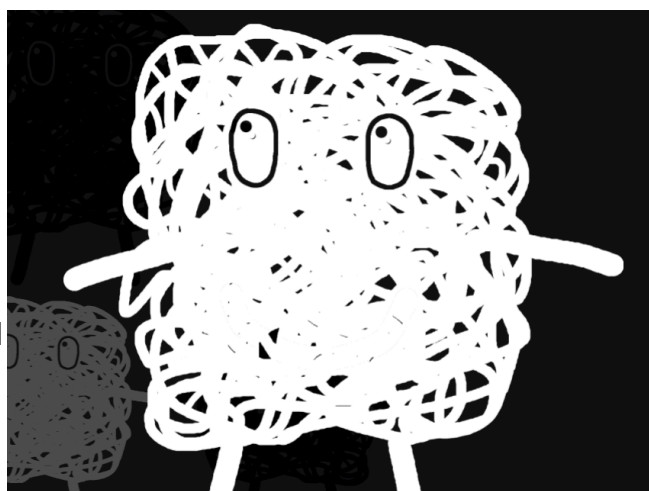
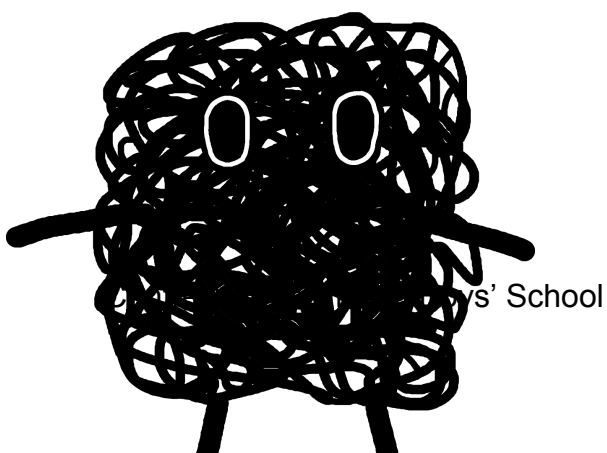
Depth first traversal

The depth-first traversal approach to traversing a tree works by exploring all the way down the tree on the left side before reaching a terminal node, then working back up the tree to go down the right side, recursively.

After careful consideration on which algorithm I should use to approach my problem, Breadth first will be the best possible option as it is guaranteed to reach each level of the directory before moving down to the next level.

Mascot - Prototype design 1

Designed by **Tommy Mansell**



Potential Programming Solutions

Python

Python is the programming language that I am most familiar with personally. It is very simple to use in comparison to lower level languages and has a huge range of public libraries available making it a very viable option for my project. Python allows for a procedural approach as well as an **Object oriented approach**. A procedural approach is beneficial because I can create processes for each component of the antivirus and run this in sequence as for each file being checked, the same processes are being repeated and recycled. On the other hand object oriented programming will allow me to create different objects that can react to different circumstances more efficiently using methods and attributes. For example creating an object called virus detection and assigning different methods for both heuristic and dictionary scanning.

pyClamd 1.0.2 is a public library that uses the ClamAV antivirus engine on multiple platforms. It is entirely open source.

C / C++

C++ would be a good language to be used for programming antivirus software. It is much lower level than languages I am currently familiar with, meaning it would be a steep learning curve for myself. However, the positive of being lower level is that C++ is able to communicate much more effectively with the CPU directly when compared to python or higher level languages. By being a lower level language, it will become more capable of exploring memory more in depth and being able to

perform more precise quarantining systems to ensure the malware does not spread. It is an object oriented language

Assembly

Assembly is useful in every way I mentioned above in C++, It is very low level and therefore can communicate very closely with the CPU.

However it is very complicated to code in, very complicated to debug and very complicated to understand especially as a novice programmer such as myself.

Java

Java is commonly used in antivirus making in order to give the project the ability to be used across multiple platforms and devices. Such as windows, IOS etc. However, because for my project I would like to stick to windows exclusively as I do not own many other platforms to assist in development and testing to ensure the compatibility will be seamless.

MySQL

MySQL is an open source relational database management system. It's SQL based and is easy to download and use through MySQL workbench. This makes it a good option for creating querying databases. Additionally mySQL is easy to connect to scripts in other languages such as python or C++

PostgreSQL

PostgreSQL is a powerful relational database management system that uses an object oriented approach. It's open source and free to download. It is more resource heavy than mySQL due to added features and its target demographic being more enterprise over user based.

MongoDB

MongoDB, unlike mySQL and PostgreSQL, is a noSQL database management system. It uses a document-oriented database design instead. It stores the data in documents that have what's known as a dynamic schema. This means that you do not have to state what the data type is before you insert it.

HTML

HTML or hypertext markup language is the standard scripting language for creating web pages that are displayed in the web browser. It is also compatible with CSS to create styling formatting methods to the standard HTML to give the webpage a better visual look, and JavaScript to program the webpage to give functionality.

HTML consists of blocks which are defined by `<block_element>`. These tags surround the data you wish to display.

Flask

Flask is a framework for python that allows for web applications to be hosted online. It provides simple tools required to build these web applications via python. It's extremely lightweight making it a good option for this project since the rest of my project is also in python.

Programming Solutions Chosen

Python

For my project, I have decided to build both the entire project in python. This is because:

- I am personally very familiar with python syntax, meaning that I can write and debug this code easier than other languages like C++ or Java that I would have to learn from the ground up. Additionally since python is an interpreted language rather than an compiled language, this means that debugging is also much easier as the code will execute until the erroneous line is encountered, rather than not executing at all which is the case for compiled languages. However, in choosing python for its better debugging capabilities, I am sacrificing overall performance as python is a higher level language than alternative languages making it slow in comparison.
- Secondly, python has lots of libraries and APIs that are compatible with it. This allows me to quickly import functionality of other applications into my program which will save me time having to code this functionality myself. An example of this is importing the mySQL plugin which allows me to directly query my databases directly from my python script rather than having to code this functionality myself.
- Additionally, python's GUI builder known as TKinter is a very polished and refined method of creating simple to use and understand graphical user interfaces.

MySQL

For this project, I have decided to use MySQL as the relational database management system responsible for the database portion of the project. I have chosen mySQL over alternative RDBMS' because:

- MySQL is very user-friendly and easy to use. It has lots of documentation and a large community of users making it very easy to pick up quickly. This is attractive to me since I have had limited past experiences with SQL, only really doing very basic querying on premade databases in SQLiteOnline
- Additionally MySQL is able to connect very easily to python via a pip installation of the connector library. This allows me to query and extract the data from the tables to be used within the program.

HTML

To create the page that informs users on how to stay safe online, I have decided to use HTML as the language to create the webpage. Over alternatives like Flask with python. I chose this because:

- I am already familiar with HTML syntax as I have used it in the past to work on old projects
- It is easy to host as it is compatible with hosting services such as Github pages making it hosted on the world wide web

Open Source useful tools

Zeek

Zeek is an open source network traffic analyzer. It can analyse traffic from web pages, **DNS servers** and Email servers. The information is stored in JSON log files. It also enables external databases and post-processing from other external softwares to be used on those JSON files. This can be applied to my project if I choose to place my antivirus online. Its implementation is written in C++

ClamAV

ClamAV is an open source antivirus toolkit. It specialised for scanning emails on gateways. However, it provides a large amount of other

features. Including a Daemon, which is essentially a background process that will run and perform various calibrated tasks. It also has a command line scanner as well as being able to be integrated with databases to provide frequent updates. It is said to scan files very quickly and can scan multiple types of files.

Additionally it can run on relatively low powered machines as it has low recommended specifications.

YARA

Yara which stands for yet another recursive algorithm is used to identify and classify malware. Its multiplatform being accessible across various operating systems. Additionally it can be run through both command line interface and through python or C scripts by using an extension.

It allows the user to create descriptions of malware, and then link that description to a specific binary or Hexadecimal + trace. Therefore if files encountered contain the trace defined earlier, they will be listed as having the same description. It uses texts or binary as fingerprints to uniquely identify the file.

To create Rules, you choose a name, and for the rule, and then define a condition in which yara will use to then compare against the file to check to see if it matches the rule or not.

It works in a boolean check, essentially it is as simple as whether or not the file being scanned contains the trace in the rule, there is no further analysis taking place; it simply looks at the raw data in typically hexadecimal.

It is comprised of 3 pieces: Meta, Strings and Condition

```
Unset
rule HelloWorldRule {
    meta:
        description = "A simple proof of concept to show yara rules"
        author = "Rajan Summan"

    strings:
        $hello = "Hello World!"

    condition:
        $hello
}
```

Meta

The Meta section contains metadata surrounding the rule, information commonly recorded here is:

- a description of what the rule does or what trace is contained within it
- The name of the person or organisation who wrote the rule
- A reference to where the rule can be found online
- The date it was created
- A hash of the rule under SHA-256

Strings

Strings in effect are variables that define what the rule looks for. This section here contains the actual fingerprint of the malware. In the example above, the string that my yara rule looks for is Hello World!. This means that any file that has "Hello World!" In it anywhere will be flagged. The variables are declared using \$

Condition

Conditions are defined instructions that the rule will follow. Currently the example rule has no conditions so it is set to check absolutely every file. However if cyber security specialists want to detect a specific piece of malware, and they know some of the properties of that malware, they can use conditions to speed up the scanning process. For example the Rule will only check files larger than a certain size, or only check files with a specific hash. This works using Boolean logic and operators

Yara in Practice

Yara rules can either be used individually or, lots of them can be compiled into one variable.

```
Python
import yara
import configparser
import os

config = configparser.ConfigParser()

config.read("config.cfg")

yara_d = config["yara_comparison_settings"]["yara_directory"]

rules_dict = {} # Initialises dictionary of rules

for yara_filename in os.listdir(yara_d): # Loops through every file in the
    directory

        if yara_filename.endswith(".yar") or yara_filename.endswith(".yara"):

            rule_filepath = os.path.join(yara_d, yara_filename) # Creates the full
            yara name of the rule

            rules_dict[yara_filename] = rule_filepath # appends the rules name and
            path
```

```
compiled_rules = yara.compile(filepath=rules_dict) # compile entire
dictionary of rules
print(compiled_rules)
```

To compile the rules under a single variable to compare a file against multiple signatures, it has to be in the form of a dictionary. The code above shows an iterative algorithm that appends each rule with the name to a dictionary, then compiles the whole thing using `yara.compile()`

I will use Notepad++ to create these yara rules and also edit them. As well as downloading repositories of them from github

Watchdog

Watchdog is a public python API library that specialises in monitoring changes in files and directories. It can be installed using PIP

Virtual Box

Virtual box is a multi-platform, free to use virtual machine made by Oracle. It allows for windows, linux and other operating systems to be emulated. It can be configured easily to be fully secure so that if something malicious is run within it, it cannot spread to the user's system, and thus I intend to use it within my project.

Additionally it also has python integration using `vboxapi` or `pyvbox`.

Cuckoo Sandbox

Cuckoo sandbox is an open source automated malware analysis system. It allows for malware to be safely executed in a secure environment. Additionally it produces reports on any effects that the malware had on the system.

It is compatible with many different operating systems within the virtual box.

TkInter

Tkinter is a GUI framework that is built directly into python, meaning no outside installation is required. It allows for simple lightweight graphical user interfaces to be created easily. It works using what's known as widgets. Widgets are the GUI elements that make up the whole interface, examples include buttons, labels and textboxes. Each one of these is known as a widget. These widgets then have to be added to the window individually using a method such as `.pack()`

PSUTIL

Psutil is a python library that gives access to system details and process details. It allows for tracking resource utilisation of CPU, RAM, Memory, Storage, network traffic and ports. This makes it useful during heuristic analysis and identifying the scale of threat that a piece of malware presents. It can be installed via pip and imported into python scripts simply.

Github pages

Github pages is a service offered by github that allows people to host websites directly from a github repository. It's completely free and

accessible to anyone so long as you have a github account. I have chosen to use github pages to host the educational page on the wide internet since it means i do not have to pay for the domain name myself

Specific Investigation Objectives

- 1. To test the efficiency and effectiveness of a dictionary based approach into malware identification using YARA rules**
- 2. To test the efficiency and effectiveness of a behavioural based approach into malware identification using virtual box**
- 3. To test the efficiency and effectiveness of a heuristic based approach into malicious process investigation on running active processes**

Specific Investigative tool Objectives

- 1. In order to let the investigation run smoothly, there must be a user interface to trigger each function and access other areas.**
 - 1.1 - There must be a button to begin a quick scan for malware
 - 1.2 - There must be a button to begin a in-depth scan for malware
 - 1.3 - There must be a button to exterminate any malware that is detected by the software
 - 1.4 - There must be a drop down menu containing the ability to close the program

1.5 - There must be a drop down menu allowing the user to access information regarding how to stay safe online and recommended practices for ensuring safety

1.6 - The user must be able to see information about the version of the scrubble software that is installed

1.7 - When malicious software is detected, a popup window will show up stating the file that's been identified and advising the user to delete the file.

1.8 - There must be a button to start the background process analysis

2. In order to ensure a thorough investigation using a dictionary approach, the anti-malware must be able to detect multiple different forms of malware. I must ensure that I have an extensive amount of yara rules checking against each file.

2.1 - Have at least 500 yara rules to detect various forms of malware including: viruses, trojans, ransomware and spyware

2.2 - The yara rules must be compiled upon startup of the application

2.2 - Keep a database containing a unique Identifier for every yara rule, its location on my local machine and its name

2.3 - Each file must be checked against all of the yara rules

2.4 - Each rule must look for specific string sequences that identify the rule as having a specific characteristic

3. To ensure a thorough investigation using a behavioural approach, the file being scanned is picked up by yara rules must be imported and tested in a secure virtual environment

3.1 - Each file must be imported into a secure sandbox on a virtual machine

3.2 - The file will be run within the virtual machine and examined for any malicious properties

3.3 - The status of the sandbox will then be returned

3.4 - If there is a significant rise of more than 60% in both CPU and RAM on the virtual machine, the threat level of the file will increase

3.5 - The virtual machine must run windows 11

3.6 - The virtual machine must be configured to be entirely blank and secure with absolutely no way for the malware to infect the host PC

3.7 - Once executed, the log files of the Virtual machine will be checked to review the events that took place. If suspicious activity is shown, it will increase the threat level of the file

4. The anti-malware must contain a quick search function

4.1 - The quick scan function must incrementally check through every file in vulnerable areas of the users device

4.2 - The time taken for the quick scan function should be kept as low as possible

5. The anti-malware must contain an in depth scan function

5.1 - The in-depth scan must incrementally check through every file on the users device

6. The anti-malware must check new files being downloaded / imported or edited on the device

6.1 - files with any activity must be assessed in a background, while the main program runs against compiled yara rules

6.2 - Checking new files should not impact performance of the users local machine by overloading the RAM

6.3 - New files being created or downloaded should be assessed against all the yara rules compiled

6.4 - If the new file is found to be malicious, it should undergo the same testing and extermination processes as files found through incremental scans.

6.5 - As the file is about to be scanned, the file will be hashed, and the name + the hashing value is stored in a text file in case further forensic analysis is required

7. Once malicious software has been identified to be a threat to the user, it must be exterminated safely

7.1 - This process should not affect any non-malicious files

7.2 - The process must isolate the malware in a quarantine directory

7.3 - The quarantine directory must be secure and encrypted

7.5 - Must inform the user as to the status of the process and whether or not deleting the malware has been successful

7.6 - If every instance of the malware has not been terminated, a warning message will be presented to the user recommending them to see a specialist.

7.7 - The malicious file must be given a hash value which is stored

8. A database will be kept to store information about the program and the host device

8.1 - The database must be a relational database normalised to third normal form

8.2 - The database will store information about threats found. This will include their name, a unique ID, a severity level and whether or not they were successfully exterminated

8.3 - The database will store information about the scans performed by the user including how long the scan took and how many files were scanned in the process

8.4 - The database will also include a structured list of all the yara rules being used by the anti-malware application giving each a unique Identifier.

8.5 - The information stored about the threats found will then be used to determine the order of rules being used to scan

9. Incorporate heuristic analysis to identify malicious processes running on both the local machine

9.1 - The algorithm must compare the CPU resources being used by the active process to the CPU usage across the entire system. If the process is using lots of processing power, the threat level of the process will increase

9.2 - The algorithm must compare how much memory is being allocated to the active process to the entire available system memory. If the process is using lots of active memory, the threat level of the process will increase

9.3 - The algorithm must run as a background threat to improve user performance by keeping the amount of resources it uses as low as possible

9.4 - The algorithm must assess the ports the active process is using while communicating over networks, and determine any suspicious ports being used. If there are multiple ports that might be used for suspicious alerted about the process.

9.5 - The algorithm must take into consideration the name of the file, if it contains suspicious wording or naming, it will increase the potential threat level of the process

9.6 - The algorithm must compare the total number of packets being sent and received by the process to the total number of packets for the whole system. If too much of the total network traffic for the system is coming from one process, it will be flagged as malicious and therefore will increase the threat level

10. Incorporate education page on social engineering and how to stay safe online

Technical Skills Required

Skill Name	Explanation
Graph/Tree Traversal	Graph/Tree traversal will have to be used to implement the scanning functions. This is because file explorer takes on the data format of a tree
Recursive Algorithms	A recursive algorithm will be used to ensure that every file is checked in the directory
Database access	Database access will be needed to store information about the status of the scan and the threats identified
Hashing	Hashing will be used on each malicious while it's being quarantined to ensure that every instance of it is cleared

Design

System Overview

As mentioned previously, my project will aim to locate, quarantine and exterminate malicious software on the users computer. The project will be split into 3 different forms of software:

- The scanning software, which will be subdivided further into a quick scan and a deep scan. Will scan every file on the user's computer and will either check the source code of the file to look for virus traces, or run the program in an isolated virtual environment and then determine if the file is malicious based on its characteristics.

- A quarantine program will then be used to ensure the virus cannot harm the computer during the process of extermination. This will be hosted on the virtual machine.
- Eventually a system will be implemented to delete the malicious file, and then re-check every instance to ensure that it has not self replicated.

Key data structures

Data structures are used in programming to organise data in an efficient way.

Hash Tables and Hash Functions

Hash tables, also known as hash maps. Are a form of data structure that takes a similar form to an array. It maps each index of the array with its specific value using a key.

A Hash Function is an algorithm that will generate the keys used to index the data within the hash table.

Trees

Trees are a subset of the graph data structure that contain no repeating cycles. As mentioned earlier, these are crucial to my project as the basis of the incremental file scan use tree traversal methods, as user directories can be modelled as a tree.

Arrays

Arrays are fixed length data structures that can store multiple pieces of data that are of the same type. The position of each element are located by an index that starts at 0. They are vital to the project as they are used to withhold the directory locations for vulnerable areas of the PC that will be checked in the quick scan function

Dictionaries

In Python, dictionaries are mutable data structures that allow you to store key-value pairs. This means that for every value you store within the database, it will have a specific key associated with it used to access that value.

I have incorporated dictionaries when compiling the yara rules, as the python API syntax requires the rules to be stored in a dictionary before they can be compiled into a single entity from a directory.

Key Algorithms

Graph and Tree traversal algorithms

Graph and Tree traversal algorithms which I implemented previously in my prototype are essential to my project. This is because my project has to be able to traverse the users folders and drives in a logical order to ensure that every file can be checked for malicious properties. The user's storage drive can be modelled as a tree starting at the root (in most cases it is C:\) and then each folder being represented as a child node, and then each file being represented as a terminal node.

For my project I will use a **breadth-first tree traversal algorithm** as it removes unnecessary backtracking which will hopefully optimise speed and efficiency of traversal.

Recursion

Recursion is the process of calling a subroutine from within itself creating some form of loop. I intend to use a recursive algorithm when designing my breadth-first traversal algorithm.

Brute force algorithm

A brute force algorithm or exhaustive search algorithm is an algorithm that systematically explores every possible option of solution to a problem until a solution is found.

I plan to implement a brute force algorithm through the use of my dictionary scan algorithm. It will systematically check each file using an array of yara rules until it finds a match, if it does not find a match it will move on to the next file.

Re-Ordering algorithm

By researching and prototyping my project using yara, I have noticed that in order to compile the rules that are going to be used for comparison, they first must be stored in a dictionary data structure. It will then move incrementally through the dictionary.

Below is the pseudo code as to how this algorithm will be implemented

Unset

Import Dictionary

Query Database to determine rules encounter the most per user

List the results from rule most used to rule least used

Convert the dictionary into a list of key-value pairs

Use the names returned from the SQL query to re-order the list of values and keys

Take the two lists and convert it back into a dictionary

IPSO Graphs (Input, Process, Storage, Output)

IPSO	Description
Input	- Files to be scanned - Yara rules used to detect malware - User inputs for beginning scans
Process	- Scan files using Yara Rules - Place suspicious files into sandbox for analysis - Quarantine flagged files into secure directory
Storage	- Store errors, events and actions in SQL database - Store malicious files found in SQL database - Store information about the scan in SQL DB

Output	• Output to Terminal • Output education page
--------	--

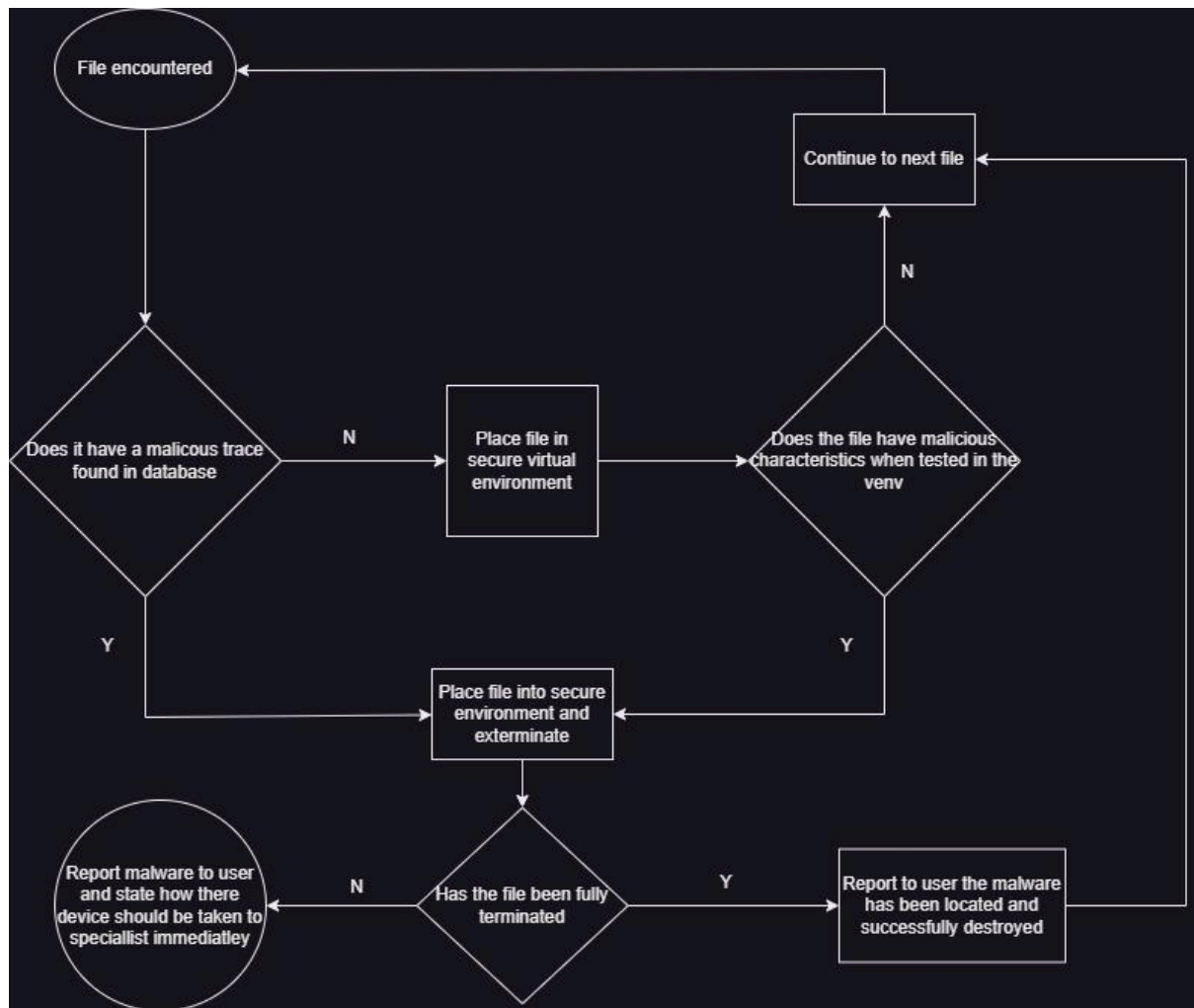
Hierarchy Charts

This graph shows how I intend to decompose the project into manageable subroutines that will be developed individually to create the final project.

Hierarchy Chart 1 - original outline

Flow Diagrams

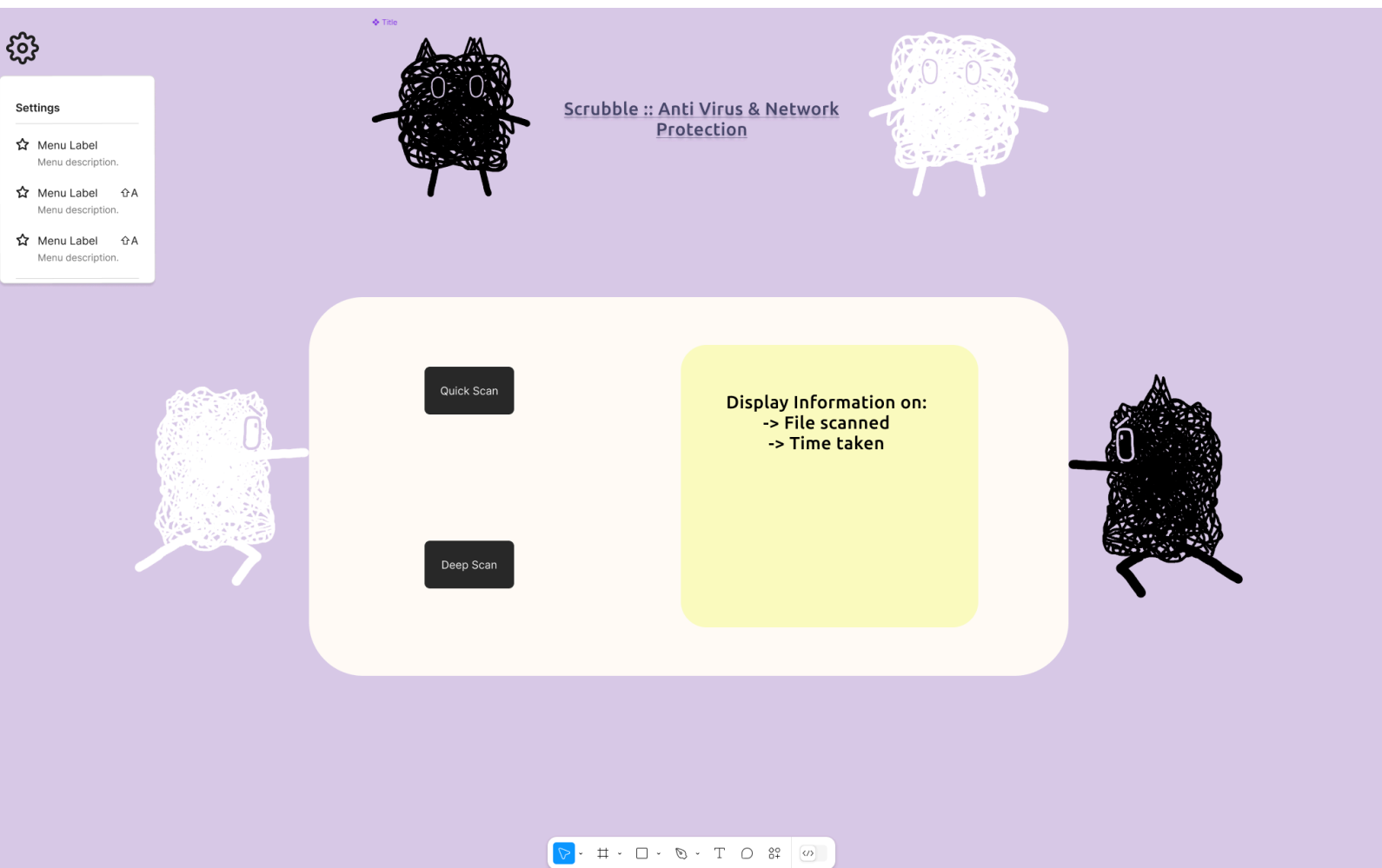
This flow diagram shows the general processing steps behind how the software will deal with each file when encountered by the traversal method.



User Interface

As mentioned previously in the project specification, I want my project to be accessible to everyone. This means that I have to make sure that my product is simple to understand and interact with. Therefore, the design of the user interface is critical to meeting this objective.

Below is a first draft of a design for my user interface.



Permanent data storage format

Database

A database is a persistent collection of organised data stored into a series of related tables. They allow for vast quantities of data to be stored and accessed quickly and reliably. This is how I intend to store traces from common malware that exist and are frequently encountered by users online.

Centre Name: Hitchin Boys' School CNO:17407

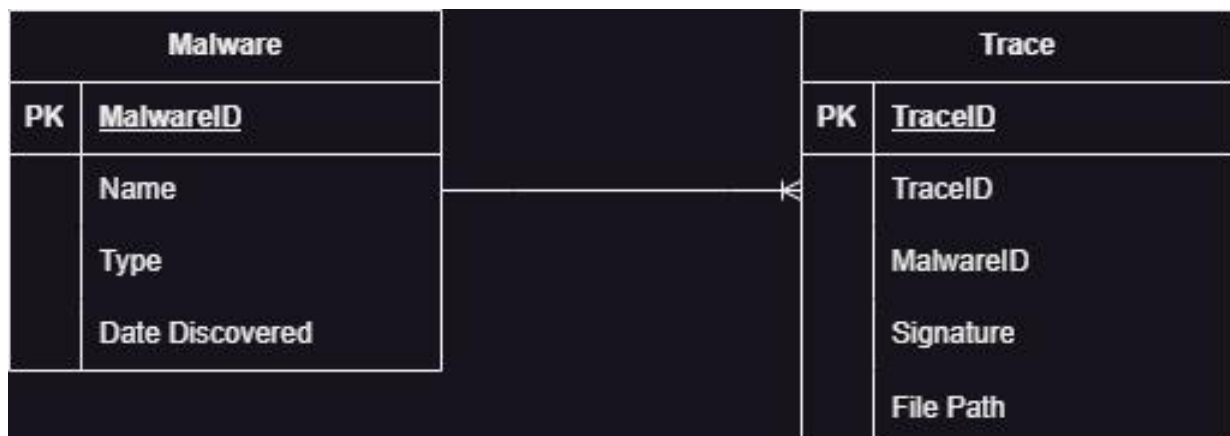
This means that I can get the best possible coverage of all forms of malware and I will be able to pick up any malicious entities quickly.

Virus trace databases can be commonly found online from sources such as Clam AV or Comodo.

The database will have to be frequently updated due to new malware being released very frequently across the world. Therefore using an open source database that is being updated regularly will be suitable.

Alternatively, a database could be used to store information about the anti-malware as a whole. This may include storing information about the threats found, such as when they were found and what specific trace identified that threat.

Example: Entity Relationship Diagram



Hash Table

A hash table is a form of data structure that stores values using an index position calculated using an associated algorithm called a hash function. They are useful for quickly inserting, searching and deleting data

There are 3 core components:

- **Key** - The value you want to store, which is fed into the hashing function to produce the index position
- **Hash Function** - a mathematical function that takes the key as an input and returns the index position for that value.
- **Hash Table** - The hash table is the data structure that stores the key values at the index positions produced by the hash function

Hashing is commonly used in encryption processing due to the nature of the algorithm being one way, meaning that it is very hard to reproduce the original value from the original key. Especially with common hashing algorithms such as **SHA-256**. This therefore may be something to use in order to secure sensitive information of users

Additionally, because hash tables have immensely fast lookup speeds, it may be useful in the quick scan function I plan to implement into the antivirus. Using a condensed version of the extensive database stored into a hash function, It will be faster to compare against and can therefore identify threats faster.

Log files

As my project deals with files being checked, moved around and being deleted constantly. I need a way of being able to keep track of what is happening. This can be done in a number of ways. For example a text file format or a database, regardless it has to be stored permanently.

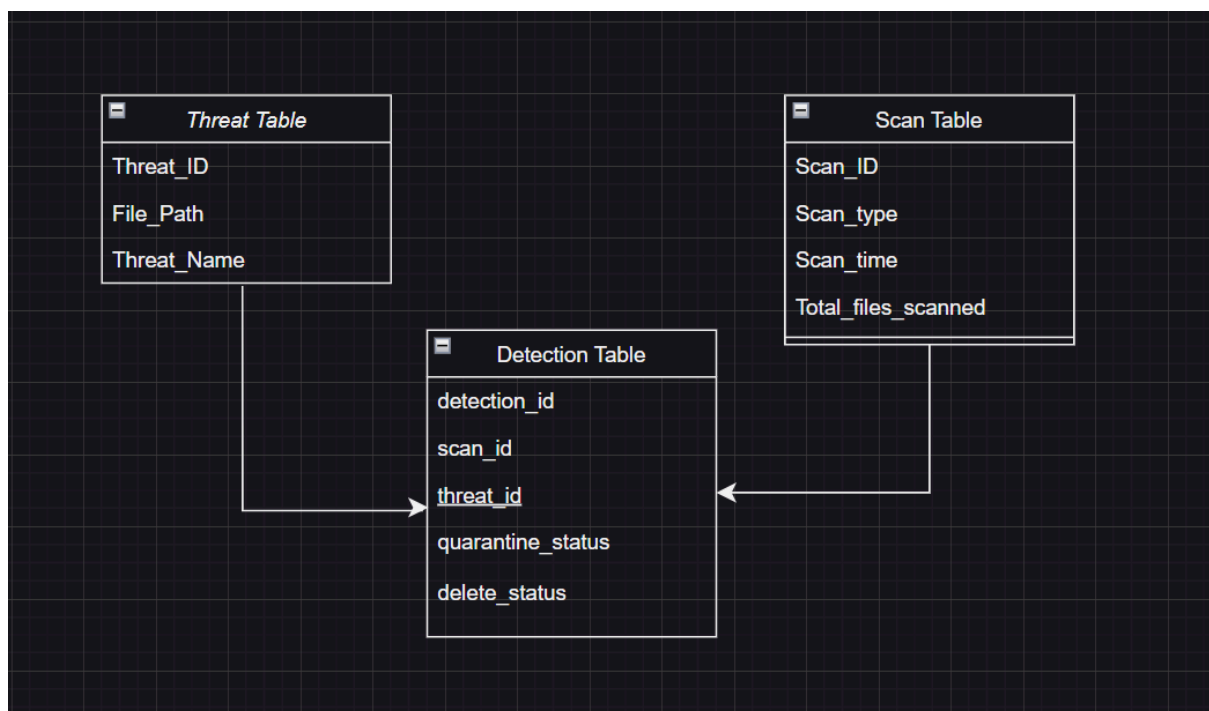
Database Design

Tables

- **Threat table** - Used to keep track of the malware that causes or presents a potential threat to the system. This is critical as it helps in reverse-engineering the attack to help specialists recover any damage.
- **Scan table** - keeps track of information surrounding the scan, such as the time taken or any errors that occurred, and a reference to the threats found
- **Quarantine table** - This table is required to keep track of the files that have been successfully quarantined, and provide a location as to where that has taken place.
- **User Table** - This table will keep track of the users name and unique ID. This way we know what user did what scan, and what malware they encountered.

Below is the Entity Relationship diagram for the database I intend to implement.

Prototype 1



Prototype 2

Configuration File

A configuration file is a file used to store important setup information that can be stored and accessed across multiple scripts.

Internal Data Storage

Object Oriented approach

Using python, OOP is a programming paradigm that allows for objects in the real world to be represented using code. This is useful in many different applications when writing code that is either about, or interacts with real world objects.

This is useful as many viruses are built using OOP as it allows for polymorphism and self replication to be done simply, therefore this allows the reverse engineering process to be refined. Additionally this allows me to create various methods to represent each function of the project. Such as a .Scan() method which will call the algorithm to traverse the storage drive. These will be stored as associated functions for a specific object

Insert Class Diagram Here

Tree

A tree is a form of **graph** that does not contain cycles. This means that the nodes follow a hierarchical structure downwards and nodes below cannot reconnect to parent nodes above

Its comprised of 3 types of nodes:

- Root node - Node at the top of the tree
- Parent nodes - Nodes that have other nodes connected below it
- Terminal / Leaf nodes - Nodes at the bottom of the tree with no other nodes connected to it.

This is useful because the user's secondary storage device can be modelled as a tree data structure as mentioned above. Meaning that they can be traversed using tree traversal algorithms to create the logic behind scanning methods within the antivirus

Priority Queue

A queue is a first in first out data structure. This means data that is added to the data structure first will be the first to be processed.

A priority queue differs slightly from this, rather than the order being decided by when the items are added to the queue. Each item is assigned a level of priority, then is added to the end of the items with the same level of priority.

This is a useful data structure for network protection. As the incoming packets can be assessed and then given a level of priority depending on the magnitude of threat the webpage causes. This can then cause the software to react differently depending on the threat level. For example, if the incoming packets are only slightly malicious and will not cause an immediate threat, my software can simply warn the user of potential danger, whereas if there is more associated threat, the software can block or blacklist the site entirely.

Stack

A stack is a last in first out data structure similar to a queue. However, here the last piece of information to be added is the first to be executed.

There are many functions around stacks that can have many functional uses within the project

- Peek - Looks at the value at the top of the stack
- Push - Adds a new value to the top of the stack
- Pop - Removes the item from the top of the stack
- IsEmpty - Checks to see if the stack is empty
- IsFull - Checks to see if the stack is full

In my project, I plan to implement this data structure to keep track of the number of files scanned. I can use a stack to push each non-malicious file onto a stack, so that when a malicious file is encountered, the peek command can be used to identify the name of the file that caused the issue and its file location.

Algorithms

Breadth first traversal

Breadth first traversal is a method of graph traversal that I am implementing into my project to be the primary logic behind the **deep file scan**. As mentioned earlier in and demonstrated in the prototype, Breadth first traversal has been selected as it removes the backtracking found in depth first and provides the quickest result for graph traversal methods.

Heuristic Algorithms

Heuristic virus identification is a crucial component in modern day antivirus software due to cyber criminals constantly creating intuitive ways to breach software. Therefore Heuristic algorithms will compile data on the behaviour of the file found within the code and create an educated guess on whether or not the file can be identified as malicious. These are performed within a virtual machine to ensure security.

For my project specifically, I intend to use Heuristic analysis in partnership with the Virtual machine. I plan to configure the virtual machine to act as a sandbox where the malicious file can be imported safely into. Then the file will be allowed to execute as it would if it were to run on a normal computer.

Once this is done, I will take measurements of the CPU utilisation and RAM used before and after the malicious program has been run. If there is a significant increase in the amount of resources being taken up by the process, it will be recorded and will increase the threat level of that file.

Additionally, VirtualBox provides detailed log files that go over the activity that takes place on the virtual machine. These file will be scanned over to look for any potential signs of malicious activity, such as the file self replicating or windows defender being activated

This will then be returned to the user.

Pseudocode

Static Heuristic analysis:

Centre Name: Hitchin Boys' School CNO:17407

Decompile subject into source code

Compare code to malicious code

 If subject source code matches malicious code over x%:

 Flag subject as malicious

Dynamic Heuristic analysis:

Place subject into virtual machine

Simulate how the subject would run as if it was on the users machine

 If command is suspicious:

 Flag subject as malicious

Pseudocode for My Project:

For File in Malware List:

 Run secure virtual machine environment

 Allow startup programs to run to simulate real life PC activity

 Take measurement of CPU and RAM utilisation

 Import Malicious File

 Allow Malicious file to execute using execute command

 Take measurement of CPU and RAM utilisation

 Close Virtual Machine

 Revert to Clean snapshot of VM

 If resources has increased by 1.6:

 Increase threat level by 1

 Scan virtualbox log files:

 If Self Replication or Windows defender has occurred:

 Increase threat level by 5

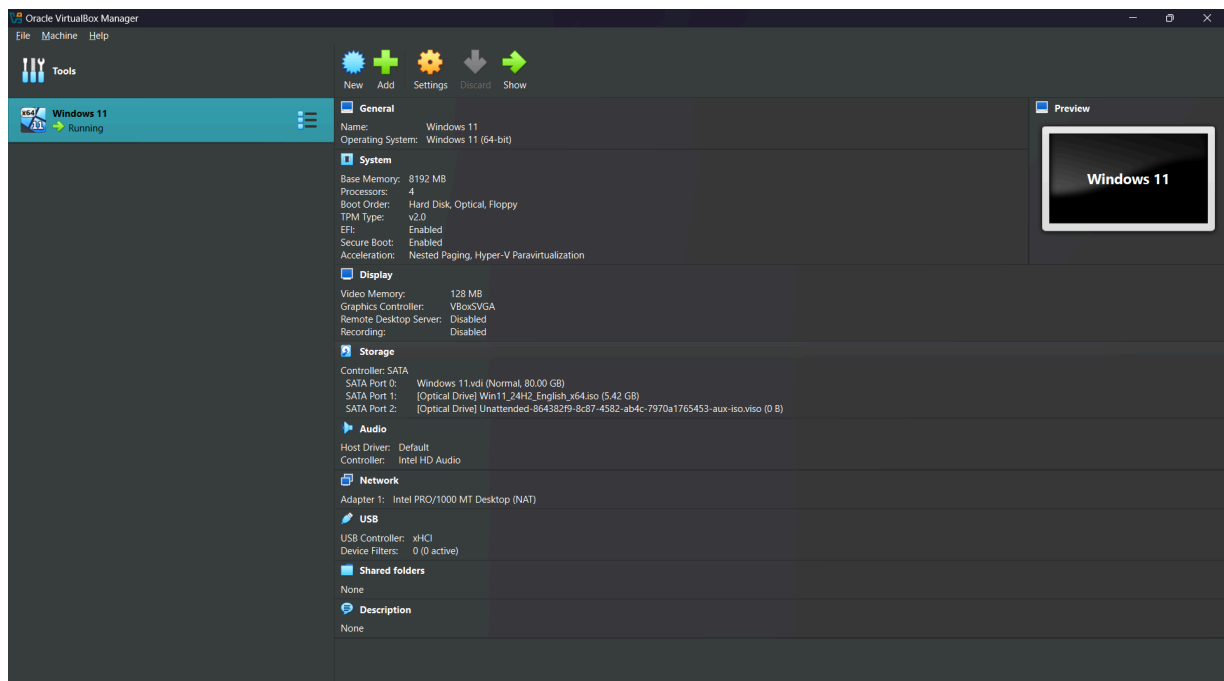
Sandboxing

Sandboxing is the virtual machine that files can be run safely in. It is entirely isolated from the user's PC so that if the file is malicious, it cannot infect the user's PC. Files can be tested safely to analyse their behaviour to identify malicious properties, such as self replication.

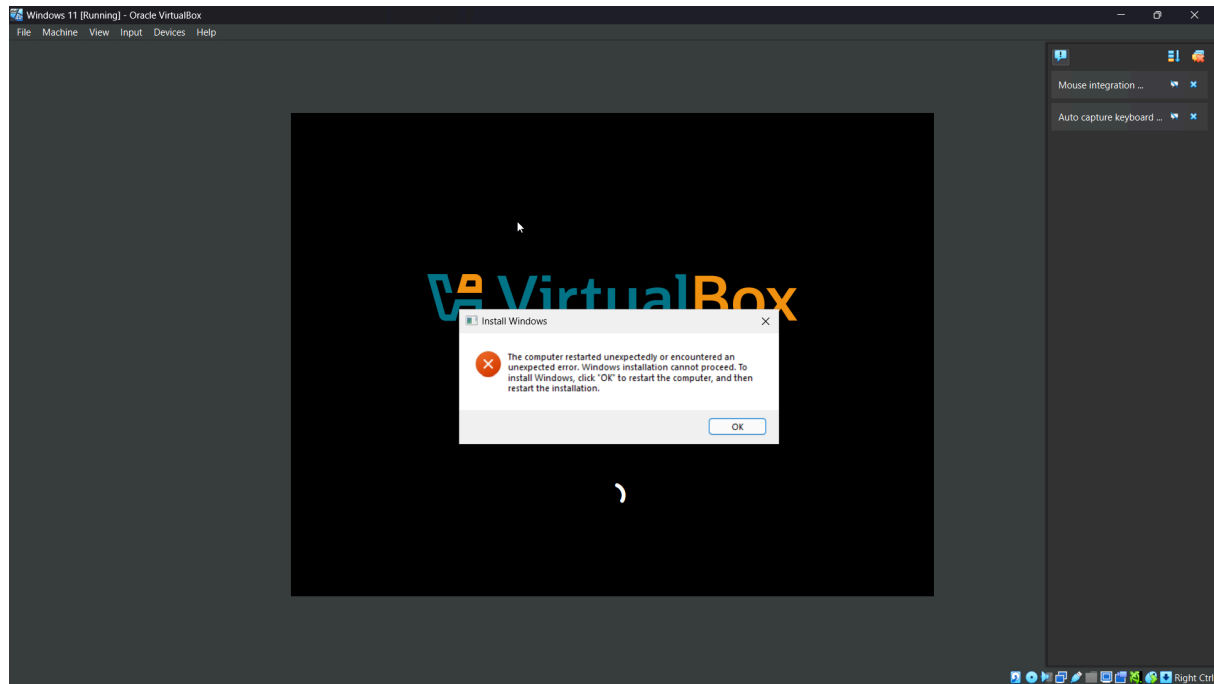
I plan to use a windows 10 iso file with Virtual box as the sandbox.

Virtual Machine Setup and Configuration

For my Virtual machine, I've decided to use Virtual box as it has the best compatibility and security options for a windows based virtual machine. It also supports a range of various operating systems.



Below shows the specifications I've set for the virtual machine.



However, I did encounter an error when trying to load windows into a virtual box. The issue occurred because the virtual machine restarts during the windows installation process.

To resolve this error, I had to edit the setup file in the registry editor to allow the installation to proceed properly.

To ensure that the Virtual machine was fully prepared for sandboxing I changed a few settings to ensure the safety of my device:

- Isolated the VM from the internet and host system by changing the network settings of the VM to use “Host-Only”
- Disabled shared folders and clipboard to prevent malware from escaping the VM
- Used an initial snapshot of the clean system to revert to after the malware has been tested.

Baseline Activity for System Processes

Add Baseline Data here

Things to look for:

Centre Name: Hitchin Boys' School CNO:17407

- Monitor CPU - psutil
 - Between 50-60%
 - Consistently high usage
 - Excessive levels of content switching
- Monitor ports - psutil, socket, scapy,
 - Look for unusual open communication ports
 - Look for unusual outbound connections
 - Look for High levels of traffic on suspicious ports
- Monitor RAM
 - >100 MB (or >10% of total memory)
 - Look for high levels of memory consumption
 - Look for processes running in suspicious locations
- Monitor hard disk
 - Look for unusual file writing. – Watchdogs
 - (More than 50MB being read and 10MB being written in 10 seconds)
 - **C:\Windows** or **C:\Program Files**

Technical Solution

Technical Skill Showed	File Found	Page Number found	Colour
Graph / Tree Traversal	Yara_Comparison.py	76, 80, 82	
Stack / Queue Operations	Yara_Comparison.py	77	
Hashing	Quarantine_Module.py	87	
Reading/Writing from text	Quarantine_Module.py	88	

files			
Single Table SQL	Yara_Comparison.py	67, 76, 79, 89	
Simple OOP	Watch_Directory()	86	
Dictionaries	Yara_Comparison.py	78	
Relational Database	Relating Databases	103, 104	
Calling Web service API	Index.html		
Files Organised for sequential access			

GUI.py

Addition of different users

Python

```
def education_page():
    webbrowser.open("https://rajansumman.github.io/Scrubble/")

    user_id = None
```

```

def select_user_number(user_number=None):

    current_user_index = 0

    def user_sql(user_number):
        try:
            cursor = connection.cursor()

            query = "SELECT user_id FROM users WHERE user_id = %s"
            cursor.execute(query, (user_number,))
            result = cursor.fetchone()

            if result:
                return result[0]
            else:
                return None
        except mysql.connector.Error as e:
            messagebox.showerror("Database Error", f"An error occurred: {str(e)}")
            return None

    if user_number is None:
        user_number = current_user_index + 1

    user_id = user_sql(user_number)

    if user_id:
        GUI_to_Yara.set_user_id(user_id)
        messagebox.showinfo("User Selected", f"User {user_number} selected (ID: {user_id})")
        current_user_index = user_number % 5 # Cycle back to 1 after User 5
    else:
        messagebox.showerror("User Selection Error", f"User {user_number} not found in the database.")

    menu_setup()
    connection.close()

```

YARA Comparison.py

The functionality of this code at an extracted level is to provide Scrubble with a dictionary type form of malware identification. This tool specifically hooks into the GUI containing both a quick and in-depth scan.

An overview of the mechanics of the dictionary search is that it takes yara rules that are stored in a dictionary, and re-arranges that dictionary to place the most common rules at the front. Then uses that dictionary of rules against each file being scanned to find any malicious properties. If there are more than 10 rules that match one file, it will be flagged as malicious.

Compile_yara_rules()

This function takes all of the yara rules I downloaded from: <https://github.com/Yara-Rules/rules>, and compiles them into a single variable. YARA incorporates this functionality using the built-in function `yara.compile()` where you pass in the file path where the rules are stored. However, the input must be in the form of a dictionary if you wish to compile multiple rules which I do, with each rule having a matching key. For this key I simply used the name of the rule.

To implement this I used a `os.walk` and a for loop to iterate through every yara rule, and appended them to the dictionary with their original file name. So it can be passed into the function `yara.compile()`.

I then returned the compiled rules from the function so that I can use them in the incremental scans.

Python

```
def compile_yara_rules(yara_directory):  
    rules_dict = {} # dictionary to store rule paths
```

```

for root, _, files in os.walk(yara_directory):
    for file in files:
        if file.endswith(".yar") or file.endswith(".yara"):
            rule_path = os.path.join(root, file)

            rule_key = os.path.relpath(rule_path, yara_directory)
            rules_dict[rule_key] = rule_path

try:
    # Compile all YARA rules from the dictionary
    compiled_rules = yara.compile(filepaths=rules_dict)
    print("YARA rules compiled successfully.")

    return compiled_rules
except yara.SyntaxError as e:
    print(f"Syntax error in YARA rules: {e}")
except Exception as e:
    print(f"Error compiling YARA rules: {e}")
return None

```

Check_yara_rule

The function below is the primary logic used to determine whether or not a file is malicious based on the yara compiled in the previous function.

Some files are unable to be scanned using yara on this device due to administrative restrictions on this device. If used in an applied cyber specialist context, these restrictions would be lifted allowing for all files to be scanned. However, since my machine does have these restrictions, I included a try except block to work around this.

The logic below dictates that a file will only be appended to the global malware list which will be transferred to the code responsible for quarantining and deleting if there are 10 or more rules that apply. I've also added some time stop between each scan to prevent overloading the CPU.

For testing reasons, I've included an If statement that specifically looks for the HelloWorldRule. This proves that a specific rule works as intended.

```
Python
def check_yara_rule(compiled_rules, file_path, file, yara_d, user_id):
    try:
        print(f"Comparing {file_path} to YARA rules")
        matches = compiled_rules.match(file_path)

        if matches:
            print(f"Matches found in {file_path}:")

            for match in matches:
                print(f"Rule name for match: {match.rule}")

                if match.rule == "HelloWorldRule":
                    print(f"Malware being appended is: {file_path} ")
                    malware.append(file_path)
                    time.sleep(3)
                    print("Malware successfully appended ", malware)

            if len(matches) >= 10:

                print(f"Malware being appended is: {file_path} ")
                malware.append(file_path)
                time.sleep(3)
                print("Malware successfully appended ", malware)

        else:
            print("While matches are present, there isnt enough of them
to warrant concern")
```

Imbedded SQL - Threat table

Below is the code to add the data from the scans to the threats table.

Once a file is identified as malicious by the yara rules it first has to identify the name of the file that contained the rule that identified the file. This is the name of the yara rule. This is used to find the related ID to be added to the database. This logic is held within the **find_yara_name** function.

First, it traverses through the directory of yara rules and looks for each file that ends in .yar or .yara. These are what identify a file as a yara file.

Then, it opens the file by joining the root of the yara directory to the name of the file. Finally it reads the content of each file and looks for the name of the match identified within the file. If it is present it returns the file name

Next, the **get_rule_id** will query the database to find the rule_id that matches the name we found earlier. It then returns this ID

Finally the **threat_sql** function is responsible for adding the correct data to the database once it has all been corrected. The data that needs to be added is, a threat_id, the file path to the threat identified, the name of the threat identified, a severity level, the yara rule ID that identified the file as malicious and the user_id

- The threat_id is created by adding 1 to the last id recorded
- The threat location and name are parsed into the function
- The severity level is determined by the number of yara rules that are being applied to the file
- The yara rule id is parsed into the function from the previous get_rule_id function
- The user_id is pulled from the GUI.py code using an in between file called GUI_to_Yara.py

Python

```
def find_yara_name(match, yara_d):
    for root, dirs, files in os.walk(yara_d):
        for file in files:
            if file.endswith(('yar', 'yara')):
```

```

        with open(os.path.join(root, file), 'r') as f:

            if f"rule {match}" in f.read():

                return file

    yara_name = find_yara_name(match.rule, yara_d)

    def get_rule_id(yara_name):
        cursor = connection.cursor()
        cursor.execute("SELECT rule_id FROM yara WHERE rule_name =
%s", (yara_name,))
        result = cursor.fetchone()[0]
        cursor.close()
        return result

    rule_id = get_rule_id(yara_name)

    def threat_sql(file_path, matches, file, rule_id, user_id):
        try:
            cursor = connection.cursor()
            cursor.execute("SELECT threat_id FROM threats ORDER BY
threat_id DESC LIMIT 1")
            previous_ThreatID = cursor.fetchone()[0]

            sql = "INSERT INTO threats (threat_id, file_path,
threat_name, severity_level, yara_rule_id, user_id) VALUES (%s, %s, %s, %s,
%s, %s)"

            val = (previous_ThreatID + 1, file_path, file,
len(matches), rule_id, user_id)

            cursor.execute(sql, val)
            connection.commit()
            print(cursor.rowcount, "record added")
            threat_ids.append(previous_ThreatID)

        except mysql.connector.Error as e:
            print(f"Error with DB: {e}")

    threat_sql(file_path, matches, file, rule_id, user_id)
else:
    print(f"No match found in {file_path}.")
except Exception as e:
    print(f"Could not scan {file_path} due to error: {e}")

```

Imbedded SQL - Scan table

This code is run after the scan is complete for both the quick and in-depth scan. It is responsible for storing the appropriate data to the scan_logs table via SQL.

```
Python
try:
    cursor = connection.cursor()
    cursor.execute("SELECT scan_id FROM scan_logs WHERE scan_id=(SELECT
max(scan_id) FROM scan_logs)")
    previous_ID = cursor.fetchone()[0]

    sql = "INSERT INTO scan_logs (scan_id, scan_type, scan_time,
total_threats_found) VALUES(%s, %s, %s, %s)"
    val = (previous_ID + 1, "quick", str(time_taken),
total_threats_found)

    cursor.execute(sql, val)
    connection.commit()
    print(cursor.rowcount, "record inserted")
    cursor.close()
    connection.close()
except mysql.connector.Error as e:
    print(f"Error with Database: {e}")
```

Re-Ordering Algorithm

Below is the algorithm I wrote to optimize the efficiency of the yara rule scanning. This algorithm uses an SQL query on the threats table to count the number of times each yara rule has been seen to identify a threat across all users. This is then returned into a priority queue data

structure, with the rules that have been used most sit at the front of the queue, and the rest behind.

Because the key for each pair in the dictionary is its unique yara_ID which is found in the dictionary, this list can be used to re-order the dictionary so that the rules used most commonly are at the front, and the rules used least are at the back.

To do this, I created a new dictionary, then iterated through the list of rule_id's found by the SQL. For each of the ID's I appended the key, value pair to the new dictionary from the old one. This created a dictionary of yara rules that matched the order of the order_list.

Then I wrote another iterative statement to loop through the entirety of the old dictionary. And if the key pair was not already in the dictionary, it would append this value to the dictionary.

This resulted in a completed dictionary, with the rules most commonly used shifted to the front in order.

Python

```
def order_dictionary(rules_dict, order_list):
    # Create ordered dictionary from the order list
    ordered_dict = {str(key): rules_dict[str(key)] for key in order_list
if str(key) in rules_dict}

    for key, value in rules_dict.items():
        if key not in ordered_dict:
            ordered_dict[key] = value

    return ordered_dict
```

GUI_to_Yara.py

This short piece of code is responsible for handling the transmission of the user_id from the GUI module to the yara_comparison module. This is important because I cannot import directly from GUI to yara because there would be a circular import error.

Therefore, by defining the user_id as a global in a separate module and assigning the id before returning it again it removes that error and allows for the data to be sent from one file to another.

Python

```
user_id = None

def set_user_id(new_user_id):

    global user_id
    user_id = new_user_id

def get_user_id():

    return user_id
```

Watch Directory.py

Watch_Directory()

The function below uses the watchdog API for python to monitor new files and files with any form of activity. The class MyEvent Handler is imported from the API.

I also imported the compiled yara rules from the Yara_Comparison.py module to compare and find the malware present.

Python

```

class MyEventHandler(FileSystemEventHandler):

    def on_any_event(self, event):

        if not event.is_directory:

            #print(f" file: {event.src_path} scanning")
            try:
                matches = compiled_rules.match(event.src_path) #
                Compares file found to compiled rules
                if matches:
                    print(f" match found {event.src_path}:")

                    for match in matches:
                        print(f" rule name: {match.rule}")
                        try:
                            malware.append(match)
                        except Exception as e:
                            print(f"MALWARE HAS BEEN FOUND, BEGIN SCAN
IMMEDIATLEY {e}")
                            #Insert popup here on GUI when i get round
                            to it

            except Exception as e:
                #print(f"cant scan {event.src_path}: because - {e}")
                pass

```

Full Code

Python

```

import threading
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
from yara_comparision import compiled_rules, malware
import time
import configparser

```

```

cfg = configparser.ConfigParser()
cfg.read("config.cfg")

def watch_directory():

    print("Control C to STOP background watching for new files !")

class MyEventHandler(FileSystemEventHandler):

    def on_any_event(self, event):

        if not event.is_directory:

            print(f" file: {event.src_path} scanning")
            try:
                matches = compiled_rules.match(event.src_path) #
Compares file found to compiled rules
            if matches:
                print(f" match found {event.src_path}:")

                for match in matches:
                    print(f" rule name: {match.rule}")
                    try:
                        malware.append(match)
                    except Exception as e:
                        print(f"MALWARE HAS BEEN FOUND, BEGIN SCAN
IMMEDIATLEY {e}")
                        #Insert popup here on GUI when i get round
to it

            except Exception as e:
                print(f"cant scan {event.src_path}: because - {e}")

p = cfg["watch_directory_settings"]["path_to_watch"]

observer = Observer()
events = MyEventHandler()

```

```
observer.schedule(events, p, recursive=True)
observer.start()

try:

    while True:
        time.sleep(1)

except KeyboardInterrupt: #Control C
    observer.stop()
    print("observer stopped")

def toggle_overwatch():
    overwatch_thread = threading.Thread(target=watch_directory)
    overwatch_thread.daemon = True
    overwatch_thread.start()
    print("currently watching New files")
```

Quarantine Module.py

Hashing Algorithm

Python

```
def hash_file(file_path):
    sha256 = hashlib.sha256()
    with open(file_path, 'rb') as f:
        while chunk := f.read(8192):
            sha256.update(chunk)
    return sha256.hexdigest()

def create_quarantine_folder(quarantine_folder):
```

```
if not os.path.exists(quarantine_folder):
    os.mkdir(quarantine_folder)
    print(f"Quarantine folder created at {quarantine_folder}")

def move_to_quarantine(file_path, quarantine_folder):
    if not os.path.isfile(file_path):
        print(f"File not found: {file_path}")
        return False

    filename = os.path.basename(file_path)

    quarantine_file_path = os.path.join(quarantine_folder, filename)

    try:
        filehash = hash_file(file_path)

        with open('hash_log.txt', 'a') as hf:
            hf.write(f"{filename}: {filehash}\n")

        shutil.move(file_path, quarantine_file_path)

        print(f"File moved to quarantine folder:
{quarantine_file_path}")
        return True
```

Virtual Machine.py

Python

Process Analysis.py

Full Code

Python

Index.html

Config.cfg

```
Unset
[yara_comparison_settings]
vulnerable_areas = C:\Users\rajan\Downloads
yara_directory = C:\Users\rajan\Desktop\NEA\Scrubble 1.0\MyRules

[sql_settings]
host=localhost
username=root
password=Monopoly2020!
database=test

[quarantine_space_settings]
quarantine_folder_path = C:\Users\rajan\Desktop\Quarantine Path

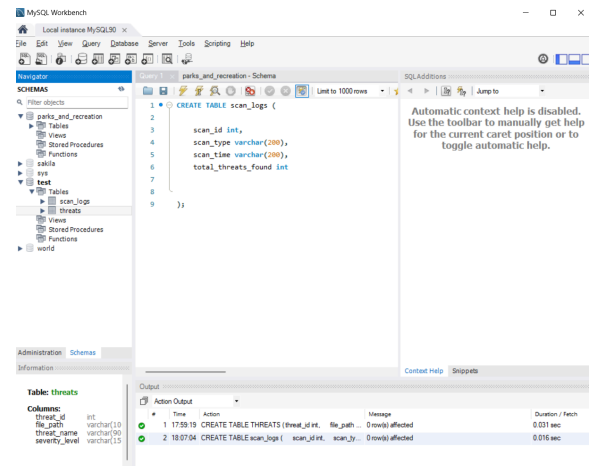
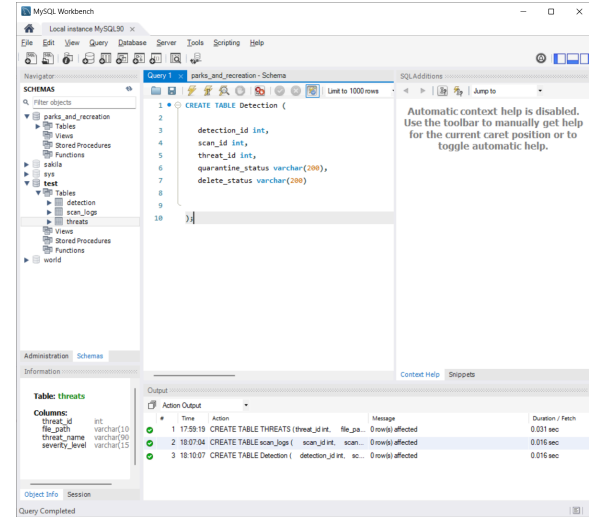
[watch_directory_settings]
path_to_watch = C:\\

[GUI_settings]
Scrubble_Man = C:\Users\rajan\Desktop\NEA\Macot Pictures\Mascot-DarkDude.png

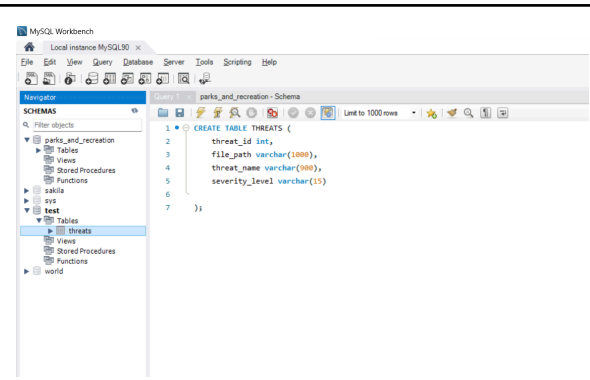
[virtual_machine_settings]
vm_name = Windows 11
guest_username = rajansumman
guest_password = Monopoly2020!
```

SQL and Database Management

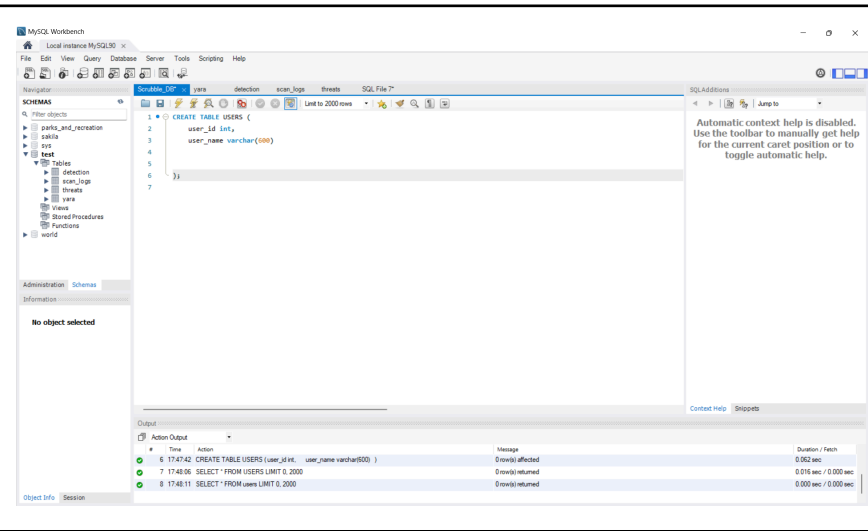
Creating databases

Name of table	Evidence
Scan_logs	 The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'park_and_recreation' database selected. The 'SQL Editor' pane shows the following SQL statement: <code>CREATE TABLE scan_logs (scan_id int, scan_type varchar(200), scan_time varchar(200), total_threats_found int);</code>. The 'Output' pane at the bottom shows the execution results: <code>1 17:59:19 CREATE TABLE THREATS (threat_id int, file_path varchar(100), severity_level varchar(15)) affected: 0 rows</code> and <code>2 18:07:04 CREATE TABLE scan_logs (scan_id int, scan_type varchar(200), scan_time varchar(200), total_threats_found int) affected: 0 rows</code>.
Detection	 The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left shows the 'park_and_recreation' database selected. The 'SQL Editor' pane shows the following SQL statement: <code>CREATE TABLE Detection (detection_id int, scan_id int, threat_id int, quarantine_status varchar(200), delete_status varchar(200));</code>. The 'Output' pane at the bottom shows the execution results: <code>1 17:59:19 CREATE TABLE THREATS (threat_id int, file_path varchar(100), severity_level varchar(15)) affected: 0 rows</code>, <code>2 18:07:04 CREATE TABLE scan_logs (scan_id int, scan_type varchar(200), scan_time varchar(200), total_threats_found int) affected: 0 rows</code>, and <code>3 18:10:07 CREATE TABLE Detection (detection_id int, scan_id int, threat_id int, quarantine_status varchar(200), delete_status varchar(200)) affected: 0 rows</code>.

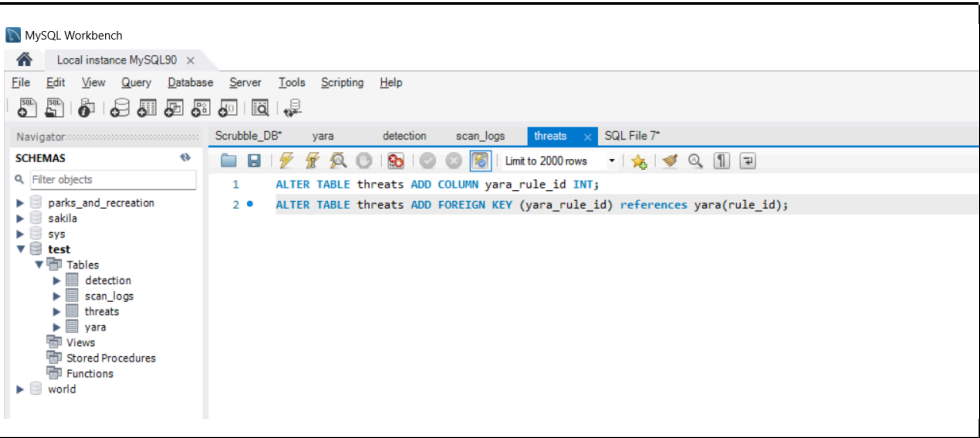
Threats

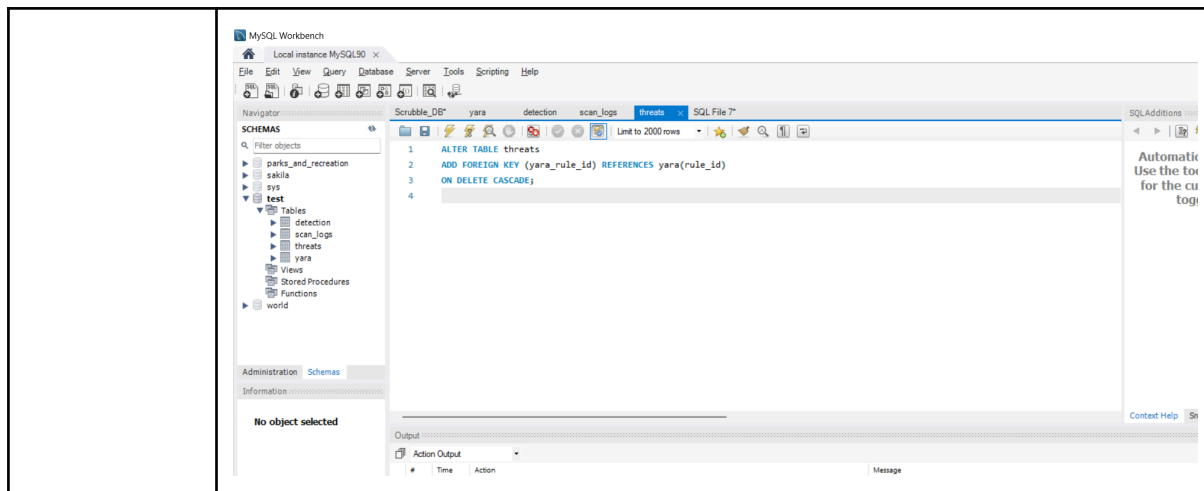


User Table



Relating Databases





SQL Queries

SQL to assign each yara rule being used a unique ID

Unset

```
INSERT INTO yara (rule_id, rule_name)
VALUES
(001, "capabilities.yar"),
(002, "crypto_signatures.yar"),
(003, "CVE-2010-0805.yar"),
(004, "CVE-2010-0887.yar"),
(005, "CVE-2010-1297.yar"),
(006, "CVE-2012-0158.yar"),
(007, "CVE-2013-0074.yar"),
(008, "CVE-2013-0422.yar"),
(010, "CVE-2015-1701.yar"),
(011, "CVE-2015-2426.yar"),
(012, "CVE-2015-2545.yar"),
(013, "CVE-2015-5119.yar"),
(014, "CVE-2016-5195.yar"),
(015, "CVE-2017-11882.yar"),
(016, "CVE-2018-4878.yar"),
(017, "CVE-2018-20250.yar"),
(018, "Android_adware.yar"),
(019, "Android_AliPay_smsStealer.yar"),
(020, "Android_Amrckr_20160519.yar"),
(021, "Android_ASSDdeveloper.yar"),
(022, "Android_AVITOMMS.yar"),
(023, "Android_Backdoor.yar"),
(024, "Android_Backdoor_Script.yar"),
(025, "Android_BadMirror.yar"),
(026, "Android_Banker_Acecard.yar"),
```

```

(027, "Android_BatteryBot_ClickFraud.yar"),
(028, "Android_Clicker_G.yar"),
(029, "Android_Copy9.yar"),
(030, "Android_DeathRing.yar"),
(031, "Android_Dectus_rswm.yar"),
(032, "Android_Dendroid_RAT.yar"),
(033, "Android_Dogspectus.yar"),
(034, "Android_fake_mario_app.yar"),
(035, "Android_FakeApps.yar"),
(036, "Android_FakeBank_Fanta.yar"),
(037, "Android_Finspy.yar"),
(038, "Android_generic_adware.yar"),
(039, "Android_generic_smsfraud.yar"),
(040, "Android_Godless.yar"),
(041, "Android_HackintTeam_Implant.yar"),
(042, "Android_Libyan_Scorpions.yar"),
(043, "Android_malware_Advertising.yar"),
(044, "Android_malware_banker.yar"),
(045, "Android_malware_ChinesePort.yar"),
(046, "Android_malware_Dropper.yar"),
(047, "Android_malware_Fake_MosKow.yar"),
(048, "Android_Malware_HackingTeam.yar"),
(049, "Android_malware_Ramsonware.yar"),
(050, "Android_malware_SMSsender.yar"),
(051, "Android_malware_Tinhvan.yar"),
(052, "Android_malware_Towelroot.yar"),
(053, "Android_malware_xbot007.yar"),
(054, "Android_malware_MalwareCertificates.yar"),
(055, "Android_mapin.yar"),
(056, "Android_Marcher_2.yar"),
(057, "Android_MazarBot_z.yar"),
(058, "Android_Metasploit.yar"),
(059, "Android_Metasploit_Payload.yar"),
(060, "Android_OmniRat.yar"),
(061, "Android_Pink_Locker.yar"),
(062, "Android_Polish_Bankbot.yar"),
(063, "Android_pornClicker.yar"),
(064, "Android_RuMMS.yar"),
(065, "Android_SandroRat.yar"),
(066, "Android_sk_bankTr.yar"),
(067, "Android_SlemBunk.yar"),
(068, "Android_SMSFraud.yar"),
(069, "Android_SpyAgent.yar"),
(070, "Android_Spynet.yar"),
(071, "Android_SpyNote.yar"),
(072, "Android_Spywaller.yar"),
(073, "Android_Switcher.yar"),
(074, "Android_Tachi.yar"),

```

```

(075, "Android_Tempting_Cedar_Spyware.yar"),
(076, "Android_Tordow.yar"),
(077, "Android_Triada_Banking.yar"),
(078, "Android_Trojan_Dendroid.yar"),
(079, "Android_Trojan_Droidjack.yar"),
(080, "Android_VikingOrder.yar"),
(081, "Android_VirusPolicia.yar"),
(082, "MALW_Retefe.yar"),
(083, "bank_rule.yar"),
(084, "davivienda.eml"),
(085, "transferencia1.eml"),
(086, "transferencial2.eml"),
(087, "attachment.yar"),
(088, "EMAIL_Cryptowall.yar"),
(089, "Email_fake_it_maintenance_bulletin.yar"),
(090, "Email_generic_phishing.yar"),
(091, "Email_PHP_Mailer.yar"),
(092, "Email_quota_limit_warning.yar"),
(093, "email_Ukraine_BE_powerattack.yar"),
(094, "extortion_email.yar"),
(095, "image.yar"),
(096, "scam.yar"),
(097, "urls.yar"),
(098, "EK_Angler.yar"),
(099, "EK_Blackhole.yar"),
(101, "EK_BleedingLife.yar"),
(102, "EK_Crimepack.yar"),
(103, "EK_Eleonore.yar"),
(104, "EK_Fragus.yar"),
(105, "EK_Phoenix.yar"),
(106, "EK_Sakura.yar"),
(107, "EK_ZeroAcces.yar"),
(108, "EK_Zerox88.yar"),
(109, "EK_Zeus.yar"),
(110, "Maldoc_APT_OLE_JSRat.yar"),
(111, "Maldoc_APT10_MenuPass.yar"),
(112, "Maldoc_APT19_CVE-2017-0199.yar"),
(113, "Maldoc_Contains_VBE_File.yar"),
(114, "Maldoc_CVE-2017-8759.yar"),
(115, "Maldoc_CVE-2017-11882.yar"),
(116, "Maldoc_CVE-2017-0199.yar"),
(117, "Maldoc_DDE.yar"),
(118, "Maldoc_Dridex.yar"),
(119, "Maldoc_hancitor_dropper.yar"),
(120, "Maldoc_Hidden_PE_file.yar"),
(121, "Maldoc_malrtf_ole2link.yar"),
(122, "Maldoc_MIME_ActiveMime_b64.yar"),
(123, "Maldoc_PDF.yar"),

```

```
(124, "Maldoc_PowerPointMouse.yar"),
(125, "Maldoc_somerules.yar"),
(126, "Maldoc_Suspicious_OLE_target.yar"),
(127, "Maldoc_UserForm.yar"),
(128, "Maldoc_VBA_macro_code.yar"),
(129, "Maldoc_Word_2007_XML_Flat_OPC.yar"),
(130, "cert_wiper.yar"),
(131, "DeltaCharlie.yar"),
(132, "general.yar"),
(133, "HotalAlfa.yar"),
(134, "IndiaAlfa.yar"),
(135, "IndiaBravo.yar"),
(136, "IndiaCharlie.yar"),
(137, "IndiaDelta.yar"),
(138, "IndiaEcho.yar"),
(139, "IndiaGolf.yar"),
(140, "IndiaHotel.yar"),
(141, "IndiaJuliett.yar"),
(142, "IndiaWhiskey.yar"),
(143, "KiloAlfa.yar"),
(144, "LimaAlfa.yar"),
(145, "LimaBravo.yar"),
(146, "LimaCharlie.yar"),
(147, "LimaDelta.yar"),
(148, "mastersig.yar"),
(149, "PapaAlfa.yar"),
(150, "RomeoAlfa.yar"),
(151, "RomeoBravo.yar"),
(152, "RomeoCharlie.yar"),
(153, "RomeoDelta.yar"),
(154, "RomeoEcho.yar"),
(155, "RomeoFoxtrot_mod.yar"),
(156, "RomeoGolf_mod.yar"),
(157, "RomeoHotel.yar"),
(158, "RomeoWhiskey.yar"),
(159, "sharedcode.yar"),
(160, "SierraCharlie.yar"),
(161, "SierraJuliettMikeOne.yar"),
(162, "SierraJuliettMikeTwo.yar"),
(163, "suicidescripts.yar"),
(164, "TangoAlfa.yar"),
(165, "TangoBravo.yar"),
(166, "UniformAlfa.yar"),
(167, "UniformJuliett.yar"),
(168, "WhiskeyAlfa.yar"),
(169, "WhiskeyBravo_mod.yar"),
(170, "WhiskeyCharlie.yar"),
(171, "WhiskeyDelta.yar"),
```

```

(172, "000_common_rules.yar"),
(173, "000_common_rules.yar"),
(174, "APT_APT1.yar"),
(175, "APT_APT10.yar"),
(176, "APT_APT15.yar"),
(177, "APT_APT17.yar"),
(178, "APT_APT29_Grizzly_Steppe.yar"),
(179, "APT_APT3102.yar"),
(180, "APT_APT9002.yar"),
(181, "APT_Backspace.yar"),
(183, "APT_Bestia.yar"),
(184, "APT_Blackenergy.yar"),
(185, "APT_Bluetermite_Emdivi.yar"),
(186, "APT_C16.yar"),
(187, "APT_Carbanak.yar"),
(188, "APT_Careto.yar"),
(189, "APT_Casper.yar"),
(190, "APT_CheshireCat.yar"),
(191, "APT_Cloudduke.yar"),
(192, "APT_Cobalt.yar"),
(193, "APT_Codoso.yar"),
(194, "APT_CrashOverride.yar"),
(195, "APT_DeepPanda_Anthem.yar"),
(196, "APT_DeputyDog.yar"),
(197, "APT_Derussi.yar"),
(198, "APT_DPRK_ROKRAT.yar"),
(199, "APT_Dubnium.yar"),
(200, "APT_Duqu2.yar"),
(201, "APT_Emissary.yar"),
(202, "APT_EnergeticBear_backdoored_ssh.yar"),
(203, "APT_eqgrp_apr17.yar"),
(204, "APT_Equation.yar"),
(205, "APT_EQUATIONGRP.yar"),
(206, "APT_fancybear_dnc.yar"),
(207, "APT_fancybear_downdelph.yar"),
(208, "APT_FiveEyes.yar"),
(209, "APT_furttim.yar"),
(210, "APT_FVEY_ShadowBrokers_Jan17_Screen_Strings.yar"),
(211, "APT_Grasshopper.yar"),
(212, "APT_Greenbug.yar"),
(213, "APT_Grizzlybear_uscert.yar"),
(214, "APT_HackingTeam.yar"),
(215, "APT_Hellsing.yar"),
(216, "APT_HiddenCobra.yar"),
(217, "APT_Hikit.yar"),
(218, "APT_Industroyer.yar"),
(219, "APT_Irontiger.yar"),
(220, "APT_Kaba.yar"),

```

```
(221, "APT_Ke3Chang_TidePool.yar"),
(222, "APT_KeyBoy.yar"),
(223, "APT_LotusBlossom.yar"),
(224, "APT_Minidionis.yar"),
(225, "APT_Mirage.yar"),
(226, "APT_Molerats.yar"),
(227, "APT_Mongall.yar"),
(228, "APT_MoonlightMaze.yar"),
(229, "APT_NGO.yar"),
(230, "APT_Oilrig.yar"),
(231, "APT_OpClandestineWolf.yar"),
(232, "APT_OPCCleaver.yar"),
(233, "APT_OpDustStorm.yar"),
(234, "APT_OpPotao.yar"),
(235, "APT_Passcv.yar"),
(236, "APT_PCclient.yar"),
(237, "APT_Pipcreat.yar"),
(238, "APT_Platinum.yar"),
(239, "APT_Poseidon_Group.yar"),
(240, "APT_Prikormka.yar"),
(241, "APT_PutterPanda.yar"),
(242, "APT_RedLeaves.yar"),
(243, "APT_Regin.yar"),
(244, "APT_RemSec.yar"),
(245, "APT_Sauron.yar"),
(246, "APT_Sauron_extras.yar"),
(247, "APT_Scarab_S cieron.yar"),
(248, "APT_Seaduke.yar"),
(249, "APT_Shamoon_StoneDrill.yar"),
(250, "APT_Snowglobe_Babar.yar"),
(251, "APT_Sofacy_Bundestag.yar"),
(252, "APT_Sofacy_Fysbis.yar"),
(253, "APT_Sofacy_Jun16.yar"),
(254, "APT_Sphinx_Moth.yar"),
(255, "APT_Stuxnet.yar"),
(256, "APT_Terracota.yar"),
(257, "APT_ThreatGroup3390.yar"),
(258, "APT_TradeSecret.yar"),
(259, "APT_Turla_Neuron.yar"),
(260, "APT_Turla_RUAG.yar"),
(261, "APT_Unit78020.yar"),
(262, "APT_UP007_SLServer.yar"),
(263, "APT_Uppercut.yar"),
(264, "APT_Waterbug.yar"),
(265, "APT_WildNeutron.yar"),
(266, "APT_Windigo_Onimiki.yar"),
(267, "APT_WoolenGoldfish.yar"),
(268, "APT_fancybear_dnc.yar"),
```

```

(269, "APT_fancybear_downdelph.yar"),
(270, "APT_FiveEyes.yar"),
(271, "APT_furttim.yar"),
(272, "APT_FVEY_ShadowBrokers_Jan17_Screen_Strings.yar"),
(273, "APT_Grasshopper.yar"),
(274, "APT_Greenbug.yar"),
(275, "APT_Grizzlylybear_uscert.yar"),
(276, "APT_HackingTeam.yar"),
(277, "APT_Hellsing.yar"),
(278, "APT_HiddenCobra.yar"),
(279, "APT_Hikit.yar"),
(280, "APT_Industroyer.yar"),
(281, "APT_Irontiger.yar"),
(282, "APT_Kaba.yar"),
(283, "APT_Ke3Chang_TidePool.yar"),
(284, "APT_KeyBoy.yar"),
(285, "APT_LotusBlossom.yar"),
(286, "APT_Minidionis.yar"),
(287, "APT_Mirage.yar"),
(288, "APT_Molerats.yar"),
(289, "APT_Mongall.yar"),
(290, "APT_MoonlightMaze.yar"),
(291, "APT_NGO.yar"),
(292, "APT_Oilrig.yar"),
(293, "APT_OpClandestineWolf.yar"),
(294, "APT_OPCleaver.yar"),
(295, "APT_OpDustStorm.yar"),
(296, "APT_OpPotao.yar"),
(297, "APT_Passcv.yar"),
(298, "APT_PCclient.yar"),
(299, "APT_Pipcreat.yar"),
(300, "APT_Platinum.yar"),
(301, "APT_Poseidon_Group.yar"),
(302, "APT_Prikormka.yar"),
(303, "APT_PutterPanda.yar"),
(304, "APT_RedLeaves.yar"),
(305, "APT_Regin.yar"),
(306, "APT_RemSec.yar"),
(307, "APT_Sauron.yar"),
(308, "APT_Sauron_extras.yar"),
(309, "APT_Scarab_S cieron.yar"),
(310, "APT_Seaduke.yar"),
(311, "APT_Shamoon_StoneDrill.yar"),
(312, "APT_Snowglobe_Babar.yar"),
(313, "APT_Sofacy_Bundestag.yar"),
(300, "MALW_adwind_RAT.yar"),
(301, "MALW_AgentTesla.yar"),
(302, "MALW_AgentTesla_SMTP.yar"),

```



```
(303, "MALW_Alina.yar"),
(304, "MALW_AlMashreq.yar"),
(305, "MALW_Andromeda.yar"),
(306, "MALW_Arkei.yar"),
(307, "MALW_Athena.yar"),
(308, "MALW_ATM>HelloWorld.yar"),
(309, "MALW_Atmos.yar"),
(310, "MALW_ATMPot.yar"),
(311, "MALW_BackdoorSSH.yar"),
(312, "MALW_Backoff.yar"),
(313, "MALW_Bangat.yar"),
(314, "MALW_Batel.yar"),
(315, "MALW_BlackRev.yar"),
(316, "MALW_BlackWorm.yar"),
(317, "MALW_Boouset.yar"),
(318, "MALW_Bublik.yar"),
(319, "MALW_Buzus_Softpulse.yar"),
(320, "MALW_CAP_HookExKeylogger.yar"),
(321, "MALW_Chicken.yar"),
(322, "MALW_Citadel.yar"),
(323, "MALW_Cloaking.yar"),
(324, "MALW_Cookies.yar"),
(325, "MALW_Corkow.yar"),
(326, "MALW_Cxpid.yar"),
(327, "MALW_Cythosia.yar"),
(328, "MALW_DDoSTf.yar"),
(329, "MALW_Derkziel.yar"),
(330, "MALW_Dexter.yar"),
(331, "MALW_DiamondFox.yar"),
(332, "MALW_DirtJumper.yar"),
(333, "MALW_Eicar.yar"),
(334, "MALW_Elex.yar"),
(335, "MALW_Elknot.yar"),
(336, "MALW_Emotet.yar"),
(337, "MALW_Empire.yar"),
(338, "MALW_Enfal.yar"),
(339, "MALW_Exploit_UAC_Elevators.yar"),
(340, "MALW_Ezcob.yar"),
(341, "MALW_F0xy.yar"),
(342, "MALW_FakeM.yar"),
(343, "MALW_FALLCHILL.yar"),
(344, "MALW_Fareit.yar"),
(345, "MALW_Favorite.yar"),
(346, "MALW_FUDCrypt.yar"),
(347, "MALW_Furtim.yar"),
(348, "MALW_Gafgyt.yar"),
(349, "MALW_Genome.yar"),
(350, "MALW_Glasses.yar"),
```

```

(351, "MALW_Gozi.yar"),
(352, "MALW_Grozlex.yar"),
(353, "MALW_Hajime.yar"),
(354, "MALW_hancitor.yar"),
(355, "MALW_Hsdfihdf_banking.yar"),
(356, "MALW_Httpsd_ELF.yar"),
(357, "MALW_IcedID.yar"),
(358, "MALW_Iexpl0ree.yar"),
(359, "MALW_IMuler.yar"),
(360, "MALW_Install11.yar"),
(361, "MALW_Intel_Virtualization.yar"),
(362, "MALW_IotReaper.yar"),
(363, "MALW_Jolob_Backdoor.yar"),
(364, "MALW_Kelihos.yar"),
(365, "MALW_KeyBase.yar"),
(366, "MALW_KINS.yar"),
(367, "MALW_kirbi_mimikatz.yar"),
(368, "MALW_Korlia.yar"),
(369, "MALW_Korplug.yar"),
(370, "MALW_Kovter.yar"),
(371, "MALW_kpot.yar"),
(372, "MALW_Kraken.yar"),
(373, "MALW_Kwampirs.yar"),
(374, "MALW_Lateral_Movement.yar"),
(375, "MALW_Lenovo_Superfish.yar"),
(376, "MALW_LinuxBew.yar"),
(377, "MALW_LinuxHelios.yar"),
(378, "MALW_LinuxMoose.yar"),
(379, "MALW_LostDoor.yar"),
(380, "MALW_LuaBot.yar"),
(381, "MALW_LuckyCat.yar"),
(382, "MALW_LURK0.yar"),
(383, "MALW_MacControl.yar"),
(384, "MALW_MacGyver.yar"),
(385, "MALW_Madness.yar"),
(386, "MALW_Magento_backend.yar"),
(387, "MALW_Magento_frontend.yar"),
(388, "MALW_Magento_suspicious.yar"),
(389, "MALW_Mailers.yar"),
(390, "MALW_marap.yar"),
(391, "MALW_MedusaHTTP_2019.yar"),
(392, "MALW_Miancha.yar"),
(393, "MALW_MiniAsp3_mem.yar"),
(394, "MALW_Mirai.yar"),
(395, "MALW_Mirai_Okiru_ELF.yar"),
(396, "MALW_Mirai_Satori_ELF.yar"),
(397, "MALW_Miscelanea.yar"),
(398, "MALW_Miscelanea_Linux.yar"),

```

```
(399, "MALW_Monero_Miner_installer.yar"),
(400, "MALW_MSILStealer.yar"),
(401, "MALW_Naikon.yar"),
(402, "MALW_Naspyupdate.yar"),
(403, "MALW_NetTraveler.yar"),
(404, "MALW_NionSpy.yar"),
(405, "MALW_Notepad.yar"),
(406, "MALW_NSFree.yar"),
(407, "MALW_Odinaff.yar"),
(408, "MALW_Olyx.yar"),
(409, "MALW_OSX_Leverage.yar"),
(410, "MALW_PE_sections.yar"),
(411, "MALW_PittyTiger.yar"),
(412, "MALW_PolishBankRat.yar"),
(413, "MALW_Ponmocup.yar"),
(414, "MALW_Pony.yar"),
(415, "MALW_Predator.yar"),
(416, "MALW_PubSab.yar"),
(417, "MALW_PurpleWave.yar"),
(418, "MALW_Pyinstaller.yar"),
(419, "MALW_Pyinstaller_OSX.yar"),
(420, "MALW_PyPI.yar"),
(421, "MALW_Quarian.yar"),
(422, "MALW_Rebirth_Vulcan_ELF.yar"),
(423, "MALW_Regsubdat.yar"),
(424, "MALW_Rockloader.yar"),
(425, "MALW_Rooter.yar"),
(426, "MALW_Rovnix.yar"),
(427, "MALW_Safenet.yar"),
(428, "MALW_Sakurel.yar"),
(429, "MALW_Sayad.yar"),
(430, "MALW_Scarhikn.yar"),
(431, "MALW_Sendsafe.yar"),
(432, "MALW_Shamoon.yar"),
(433, "MALW_Shifu.yar"),
(434, "MALW_shifu_shiz.yar"),
(435, "MALW_sitrof_fortis_scar.yar"),
(436, "MALW_Skeleton.yar"),
(437, "MALW_Spora.yar"),
(438, "MALW_Sqlite.yar"),
(439, "MALW_Stealer.yar"),
(440, "MALW_Surtr.yar"),
(441, "MALW_T5000.yar"),
(442, "MALW_Tedroo.yar"),
(443, "MALW_Tinba.yar"),
(444, "MALW_TinyShell_Backdoor_gen.yar"),
(445, "MALW_Torte_ELF.yar"),
(446, "MALW_TreasureHunt.yar"),
```

```

(447, "MALW_TrickBot.yar"),
(448, "MALW_TRITON_HATMAN.yar"),
(449, "MALW_TRITON_ICS_FRAMEWORK.yar"),
(450, "MALW_Trumpbot.yar"),
(451, "MALW_Upatre.yar"),
(452, "MALW_Urausy.yar"),
(453, "MALW_Vidgrab.yar"),
(454, "MALW_viotto_keylogger.yar"),
(455, "MALW_Virut_FileInfector_UNK_VERSION.yar"),
(456, "MALW_Volgmer.yar"),
(457, "MALW_Wabot.yar"),
(458, "MALW_Warp.yar"),
(459, "MALW_Wimmie.yar"),
(460, "MALW_xDedic_marketplace.yar"),
(461, "MALW_XHide.yar"),
(462, "MALW_XMRIG_Miner.yar"),
(463, "MALW_XOR_DDos.yar"),
(464, "MALW_Yayih.yar"),
(465, "MALW_Yordanyan_ActiveAgent.yar"),
(466, "MALW_Zegost.yar"),
(467, "MALW_Zeus.yar"),
(468, "POS.yar"),
(469, "POS_Bernhard.yar"),
(470, "POS_BruteforcingBot.yar"),
(471, "POS_Easterjack.yar"),
(472, "POS_FastPOS.yar"),
(473, "POS_LogPOS.yar"),
(474, "POS_MalumPOS.yar"),
(475, "POS_Mozart.yar"),
(476, "RANSOM_.CRYPTXXX.yar"),
(477, "RANSOM_777.yar"),
(478, "RANSOM_acroware.yar"),
(479, "RANSOM_Alpha.yar"),
(480, "RANSOM_BadRabbit.yar"),
(481, "RANSOM_Cerber.yar"),
(482, "RANSOM_Comodosec.yar"),
(483, "RANSOM_Crypren.yar"),
(484, "RANSOM_Cryptolocker.yar"),
(485, "RANSOM_CryptoNar.yar"),
(486, "RANSOM_DMALocker.yar"),
(487, "RANSOM_DoublePulsar_Petya.yar"),
(488, "RANSOM_Erebus.yar"),
(489, "RANSOM_GoldenEye.yar"),
(490, "RANSOM_GPGQwerty.yar"),
(491, "RANSOM_jeff_dev.yar"),
(492, "RANSOM_locdoor.yar"),
(493, "RANSOM_Locky.yar"),
(494, "RANSOM_Maze.yar"),

```

```
(495, "RANSOM_MS17-010_Wannacrypt.yar"),
(496, "RANSOM_PetrWrap.yar"),
(497, "RANSOM_Petya.yar"),
(498, "RANSOM_Petya_MS17_010.yar"),
(499, "RANSOM_Pico.yar"),
(500, "RANSOM_Revix.yar"),
(501, "RANSOM_SamSam.yar"),
(502, "RANSOM_Satana.yar"),
(503, "RANSOM_screenlocker_5h311_1nj3c706.yar"),
(504, "RANSOM_Shiva.yar"),
(505, "RANSOM_shrug2.yar"),
(506, "RANSOM_Sigma.yar"),
(507, "RANSOM_Snake.yar"),
(508, "RANSOM_Stampado.yar"),
(509, "RANSOM_termite.yar"),
(510, "RANSOM_TeslaCrypt.yar"),
(511, "RANSOM_Tox.yar"),
(512, "RAT_Adwind.yar"),
(513, "RAT_Adzok.yar"),
(514, "RAT_Asyncrat.yar"),
(515, "RAT_BlackShades.yar"),
(516, "RAT_Bolonyokte.yar"),
(517, "RAT_Bozok.yar"),
(518, "RAT_Cerberus.yar"),
(519, "RAT_Crimson.yar"),
(520, "RAT_CrossRAT.yar"),
(521, "RAT_CyberGate.yar"),
(522, "RAT_DarkComet.yar"),
(523, "RAT_FlyingKitten.yar"),
(524, "RAT_Gh0st.yar"),
(525, "RAT_Gholee.yar"),
(526, "RAT_Glass.yar"),
(527, "RAT_Havex.yar"),
(528, "RAT_Hizor.yar"),
(529, "RAT_Indetectables.yar"),
(530, "RAT_Inocnation.yar"),
(531, "RAT_jRAT.yar"),
(532, "RAT_Meterpreter_Reverse_Tcp.yar"),
(533, "RAT_Nanocore.yar"),
(534, "RAT_NetwiredRC.yar"),
(535, "RAT_Njrat.yar"),
(536, "RAT_Orcus.yar"),
(537, "RAT_PlugX.yar"),
(538, "RAT_PoetRATDoc.yar"),
(539, "RAT_PoetRATPython.yar"),
(540, "RAT_PoisonIvy.yar"),
(541, "RAT_Ratdecoders.yar"),
(542, "RAT_Sakula.yar"),
```

```
(543, "RAT_ShadowTech.yar"),
(544, "RAT_Shim.yar"),
(545, "RAT_Terminator.yar"),
(546, "RAT_xRAT.yar"),
(547, "RAT_xRAT20.yar"),
(548, "RAT_Xtreme.yar"),
(549, "RAT_ZoxPNG.yar"),
(550, "T00LKIT_Chinese_Hacktools.yar"),
(551, "T00LKIT_Dubruite.yar"),
(552, "T00LKIT_exe2hex_payload.yar"),
(553, "T00LKIT_FinFisher_.yar"),
(554, "T00LKIT_Gen_powerkatz.yar"),
(555, ".gitKeep.gitkeep"),
(556, "Javascript_exploit_and_obfuscation.yar"),
(557, "JJencode.yar"),
(558, "packer.yar"),
(559, "packer_compiler_signatures.yar"),
(560, "peid.yar"),
(561, "tweetable-polyglot-png.yar"),
(562, "base64.yar"),
(563, "domain.yar"),
(564, "ip.yar"),
(565, "magic.yar"),
(566, "README.yar"),
(567, "suspicious_strings.yar"),
(568, "url.yar"),
(569, "virustotal.yar"),
(570, "WShell_APT_Laudanum.yar"),
(571, "WShell_ASPIXpy.yar"),
(572, "WShell_ChinaChopper.yar"),
(573, "Wshell_ChineseSpam.yar"),
(574, "WShell_Drupalgeddon2_icos.yar"),
(575, "WShell_fire2013.yar"),
(576, "WShell_PHP_Anuna.yar"),
(577, "WShell_PHP_in_images.yar"),
(578, "WShell_THOR_Webshells.yar")
(579, "HelloWorldRule.yara") #Test pilot rule will be removed
```

SQL to initialise table of scan content

Python

```
import mysql.connector

def sqlconnection():
    connection = mysql.connector.connect(
        host="localhost",
        user="root", # Use 'user' instead of 'username'
        password="Monopoly2020!",
        database="test"
    )

    if connection.is_connected():
        print("Connected to SQL server successfully")

    return connection

# Create a connection and cursor
connection = sqlconnection()
mycursor = connection.cursor()

# SQL query and values
sql = "INSERT INTO scan_logs (scan_id, scan_type, scan_time,
total_threats_found) VALUES (%s, %s, %s, %s)"
val = ("000", "initialise", "initialise", 0) # Changed 000 to "000" as a
string

# Execute the query and commit
mycursor.execute(sql, val)
connection.commit() # Added parentheses to commit()

# Confirm insertion
print(mycursor.rowcount, "record inserted")

# Close the cursor and connection
mycursor.close()
connection.close()
```

Virtual Machine

Installing Virtual Box

To ensure that the virtual machine worked effectively and most importantly securely. I needed to install a lot of software to be able to use the service correctly.

Below is a list of all the software that I needed and links to where I installed it from.

- **VirtualBox**
 - <https://www.virtualbox.org/wiki/Downloads> - Simply followed the installation instructions provided
- **Windows 11 ISO file**
 - <https://www.microsoft.com/en-us/software-download/windows11> - Downloaded the correct ISO file
- **VirtualBox software development kit**
 - The virtualbox software development kit was also installed through ORACLE website. It allows me to manipulate certain areas of the virtual machine within my code. Integrating this into my code was as simple as using a pip installation of a virtual box.
- **Guest Additions**
 - Guest Additions is a collection of drivers that allow the OS running within the Virtual Machine to add more functionality.
 - The primary functionality that required me to add Guest Additions was allowing for File transfer between the Host and VM.
- **VboxManage**
 - VBoxManage is an additional tool that allows for the control and configuration of the VM from the command line.

Configuring Virtual Box

CPU and RAM

Virtual machines require the user to assign a specific amount of Processing power and RAM from the host machine. While at first I wanted to configure the Virtual machine to use as little resources of the

host PC as possible to try and keep the users machine running slowly with my application in the background.

However, after researching malware anti-analysis functionality. I learned that malware developers will program their code to check for certain things that suggest that the program is running within a virtual sandbox environment. One of the flags is if it's running on a PC with low CPU cores and RAM.

Therefore, I have decided to assign **8GB of RAM** to the Virtual machine and **2 CPU cores**.

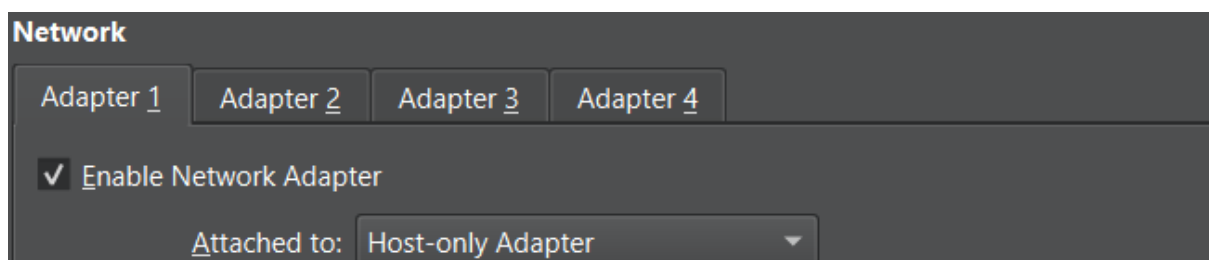
Networking

Additionally, it's also really important to Isolate the Virtual Machine from the wider internet during malware analysis and testing. This is for a variety of reasons:

- Avoid spreading to other systems
- Avoid spreading to host system
- Prevent data leaking

Therefore, I configured the networking settings so that its “Host Only” meaning that it is not connected to anything external via any forms of network.

Snapshots



Virtual box Automated code

```

Python
import subprocess
import configparser
import os
from yara_comparison import malware

def virtualbox_testing():

    config = configparser.ConfigParser()
    config.read("config.cfg")

    def startup(vm_name):

        status = ["VBoxManage", "showvminfo", vm_name]
        print(status)

        start_cmd = ["VBoxManage", "startvm", vm_name, "--type", "headless"]

        try:
            subprocess.run(start_cmd, check=True)
            print("VM Loaded successfully :) ")
        except Exception as e:
            print(e)

    def poweroff():
        poweroff_cmd = ["VBoxManage", "controlvm", vm_name, "poweroff"]
        subprocess.run(poweroff_cmd, check=True)

    vm_name = config["virtual_machine_settings"]["vm_name"]
    guest_username = config["virtual_machine_settings"]["guest_username"]
    guest_password = config["virtual_machine_settings"]["guest_password"]

    startup(vm_name)

    for file in malware:

```

```

host_file_path = file
file_name = os.path.basename(file)

guest_file_path = "C:/Users/Public/Malware_Tests/" + file_name

copy_cmd = [
    "VBoxManage", "guestcontrol", vm_name, "copyto", host_file_path,
    "--target-directory", guest_file_path,
    "--username", guest_username,
    "--password", guest_password,
]

try:
    subprocess.run(copy_cmd, check=True)
    print(f"File {host_file_path} successfully copied to {guest_file_path} on the VM.")
except subprocess.CalledProcessError as e:
    print("An error occurred:", e)

run_cmd = [
    "VBoxManage", "guestcontrol", vm_name, "run",
    "--exe", guest_file_path,
    "--username", guest_username,
    "--password", guest_password,
    "--wait-stdout", "--wait-stderr",
    "--", guest_file_path # Hopefully error checks ???
]

try:
    subprocess.run(run_cmd, check=True)
    print(f"File {guest_file_path} successfully executed on the VM")
except subprocess.CalledProcessError as e:
    print("error occurred while executing the file :", e)

poweroff()

```

Testing

Test Plan

ID	Area	Description	Test data	Expected results	Objectives shown	Time Stamp
01	GUI	Check that there are dedicated buttons to trigger the events of the antivirus	N/A	Message confirming that each function has started successfully	1.1, 1.2, 1.3, 1.8	
02	GUI	Check that there is a drop down menu to access additional information about the project	N/A	The file tab allows the user to close the program The help tab outputs the version of the software The Education tab takes the user to the dedicated page on social engineering via the users browser	1.4, 1.5, 1.6	
03	GUI	When a malicious file is encountered, a popup will show up on the GUI	HelloWorld.py & HelloWorld.txt HelloWorld.yara Other non malicious files	popup window to be displayed The name of the file included in the popup A small message advising the user on what steps to take next on the popup	1.7	
04	Yara_comparison	At least 500 yara rules are in use and compiled upon startup of the application	https://github.com/Yara-Rules/rules MySQL	A message displayed on the terminal saying that all yara rules have been successfully compiled when the application is first launched Use MySQL to show how many total yara rules are in use	2.1, 2.2	

05	Yara_comparison	A database is kept and maintained containing the name, file location and unique identifier for each rule in use	https://github.com/Yara-Rules/rules SQL - select * from yara	A table is outputted on the MySQL interface showing the relevant information	2.2	
06	Yara_comparison	Every file being checked must be analysed against the compiled yara rules	Quick scan In-depth Scan Background watch	When the quick scan button is pressed, the name of each file is outputted and a message to verify that it has been compared to the rules When the in-depth scan button is pressed, the name of each file is outputted and a message to verify that it has been compared to the rules After Startup, any files identified by the background analyser are compared to the rules, the name and a verification message will also be outputted	2.3	
07	Quick Scan	When the quick scan button is pressed, every file in a pre-defined list of vulnerable directories are checked against the yara rules.	Quick Scan C:/Users/rajan/Downloads	The areas that are considered vulnerable will be outputted to the terminal The name of each file will be outputted to check that the traversal algorithm does check every file	4.1	
08	Quick Scan	When a malicious file is identified, the name of the file will be returned as well as the yara rule where the match was found	HelloWorld.yara HelloWorld.txt HelloWorld.py Other non malicious files	The name of the file will be returned as well as the yara rule where the match was found. This will be printed to the terminal	2.3	
09	Quick	The time taken for	Quick	The time taken for the	4.2	

	scan	the quick scan must be kept as low as possible	Scan C:/Users/rajan/Download	quick scan will be outputted to the terminal The time must be below 5 minutes		
10	In-depth scan	The In-Depth scan must incrementally check through every file on the user's device. Here, the algorithm is designed to be more thorough than efficient.	InDepth scan C:\	The name of each file will be outputted to check that the traversal algorithm does check every file	2.3, 5.1	
11	Overwatch	Files with any form of activity will be assessed in a background thread against compiled yara rules	HelloWorld.yara HelloWorld.txt Other non malicious files	The name of the file will be returned as well as the yara rule where the match was found. This will be printed to the terminal If a malicious file is found, it will be appended to the overall malware list which will be shown using debug mode in VsCode	6.1, 6.3, 6.4	
12	Overwatch	Before and after startup, I will check resource monitor on my local machine to ensure an excessive amount of RAM is not being used	Task manager	Less than 5% increase in total RAM	6.2	
13	Quarantine	The directory used to delete the malicious files must be secure and encrypted so that it isolated the malicious file and does not delete any non-malicious files	HelloWorld.txt HelloWorld.py Other Non malicious files	A statement validating that the quarantine folder has been created outputted to the terminal A statement validating that the malicious file has been successfully moved to the quarantine file A statement to tell the	7.1, 7.2, 7.3, 7.6, 7.5	

				user whether or not the file has been deleted successfully If the file has failed to be deleted successfully a popup message will show up advising the user to see a specialist Immediately		
14	Quarantine	There must be secondary validation to ensure that every instance of the file has been deleted	HelloWorld.txt HelloWorld.py Other non malicious files	A statement will be outputted to the terminal stating that a secondary check has begun	7.4	
16	Quarantine	The file being quarantine must be hashed, and the name of the file and the consequent hash value is stored in a text file	hash_log.txt	I will check the text file before and after the quarantine process has taken place to show that the hashing and writing to the text file has taken place.	7.7	
17	Database	The database must be a relational database normalised to third normal form	MySQL Threat Table Scan Log Table Yara table	This will be proven using a join query in mySQL to show the relations between the 3 tables	8.1	
18	Database	A threats table will store information about threats found. This includes a UniqueID, the file path to the suspicious file and the files name, as well as the severity level	MYSQL HelloWorld.py HelloWorld.txt	Using SELECT * FROM threats, mySQL will display the information after the scanning function has been tested and the correct Hello World test files have been identified	8.2, 8.4	
19	Database	A scan_log table will store information about the status of the scan. This includes a	mySQL	Using SELECT * FROM threats, mySQL will display the information after the scanning function has been tested and the correct Hello	8.3	

		UniqueID, the time taken, and how many total threats were identified		World test files have been identified		
20	Process analysis	When the user selects the option to scan running processes by pressing the button, it must not cause a rise in RAM to prevent too much resource from being used.	Resource Monitor	Running the program and watching the resource monitor to watch for the increase in RAM and CPU percentage	9.3	
21	Process analysis	The algorithm must compare the resources (CPU, RAM) being used by the running process (programs) to the overall system.	Ultrakill (high resource heavy program)	I'll execute a program that takes up the required amount of CPU and Memory to trigger the event. This will result in the name of the process being highlighted in the terminal	9.1, 9.2	
22	Process analysis	The algorithm must look at the ports that the process is using and highlight potentially suspicious ports that are open from a list	Current system processes	Each potentially malicious port will be printed for each process	9.4	
23	Process analysis	The algorithm must compare packets in and out per process to the whole system to determine maliciousness	Network traffic simulation I	A message will pop up stating the name of the application with the excessive levels of traffic.	9.6,	
24	Education page	Display a website with useful information on social engineering and safe practices online	https://rajansumman.github.io/Scrubble/	The website will be displayed when the button is pressed	10.0	

Conclusions of the Investigation

Malware Identification methods

References

- <https://www.security.org/antivirus/how-does-antivirus-work/>,
- <https://www.cisa.gov/news-events/news/understanding-anti-virus-software>
- https://en.wikipedia.org/wiki/Category:Types_of_malware
- <https://www.imperva.com/learn/application-security/social-engineering-attack/>
- https://www.cisco.com/c/en_uk/products/security/firewalls/what-is-a-firewall.html
- <https://www.indeed.com/career-advice/career-development/types-of-encryption>
- <https://www.ncsc.gov.uk/guidance/what-is-an-antivirus-product>
- <https://www.tutorialspoint.com/graphs-and-its-traversal-algorithms>
- <https://pypi.org/project/clamd/>
- <https://docs.python.org/3/library/tkinter.html>
- <https://zeek.org/>
- <https://docs.clamav.net>
- https://www.researchgate.net/publication/377364400_Cyber_Security_using_Data_Structures#
- <https://www.techtarget.com/searchsecurity/definition/one-time-pad>
- <https://www.comodo.com/home/internet-security/updates/vdp/database.php>
- <https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>
- <https://docs.fortinet.com/document/fortigate/5.4.0/cookbook/933251/traffic-shaping-priority-queueing-priq>
- <https://www.geeksforgeeks.org/hash-table-data-structure/>
- <https://www.toppr.com/guides/computer-science/computer-fundamentals/utility-software/antivirus-software/#:~:text=Heuristic%2DBased%20Detection.can%20also%20detect%20new%20viruses.>

- <https://cyberpedia.reasonlabs.com/EN/heuristic.html#:~:text=Heuristic%20is%20a%20technique%20employed,enough%20to%20detect%20unknown%20threats.>
- https://en.wikipedia.org/wiki/Heuristic_analysis
- <https://www.geeksforgeeks.org/graph-data-structure-and-algorithms/>
- <https://www.geeksforgeeks.org/introduction-to-tree-data-structure/>
- <https://pypi.org/project/pyclamav/>
- <https://yara.readthedocs.io/en/latest/>
- <https://virustotal.github.io/yara/>
- <https://learn.microsoft.com/en-us/defender-endpoint/schedule-antivirus-scans>
- <https://support.kaspersky.com/KAV/2020/en-us/70775.htm>
- <https://www.youtube.com/watch?v=35Exd9GrR5I>
- <https://www.youtube.com/watch?v=gEqStsO7aBw&t=127s>
- <https://www.petergirinus.com/blog/how-to-install-yara-malware-analysis-tool-on-windows>
- <https://yara.readthedocs.io/en/latest/yarapython.html>
- <https://github.com/Neo23x0/signature-base>
- https://www.youtube.com/watch?v=zVry61Lg_Ic
- <https://en.wikipedia.org/wiki/SHA-2>
- <https://python-watchdog.readthedocs.io/en/stable/>
- <https://github.com/Yara-Rules/rules>
- <https://realpython.com/intro-to-python-threading/>
- <https://www.simplilearn.com/dictionary-in-python-article#:~:text=In%20Python%20C%20dictionaries%20are%20mutable,elements%20using%20the%20methods%20dict.>
- <https://www.redhat.com/en/topics/virtualization/what-is-a-virtual-machine>
- <https://docs.python.org/3/library/subprocess.html>
- <https://www.virtualbox.org/wiki/Downloads>
- <https://www.gov.uk/government/statistics/cyber-security-breaches-survey-2024/cyber-security-breaches-survey-2024>
- <https://www.ownyouronline.govt.nz/personal/know-the-risks/why-online-security-is-important/>
- <https://activesocialcare.com/handbook/safeguarding-children/the-risks-associated-with-the-internet-and-online-social-networking/>
- <https://explodingtopics.com/blog/cyberbullying-stats>
- <https://www.csis.org/programs/strategic-technologies-program/significant-cyber-incidents>
- <https://www.proofpoint.com/uk/threat-reference/social-engineering>
- <https://thecyphere.com/blog/social-engineering-statistics/>
- <https://www.kaspersky.com/resource-center/preemptive-safety/top-10-preemptive-safety-rules-and-what-not-to-do-online>
- <https://www.titanfile.com/blog/cyber-security-tips-best-practices/>
- <https://utech.co/blog/our-top-recommendations-for-protecting-your-tech/>
- <https://www.upguard.com/blog/how-to-use-nmap>

- https://www.youtube.com/results?search_query=how+to+use+nmap+on+windows
- <https://securityscorecard.com/blog/navigating-the-risks-of-tcp-445-strategies-for-secure-network-communication/>
- <https://www.ssh.com/academy/ssh/port>
- <https://www.speedguide.net/port.php?port=53>
- <https://www.varonis.com/blog/how-to-use-wireshark>
- <https://www.opensourceforu.com/2024/07/using-python-to-build-a-network-and-port-scanning-tool/>
- <https://kevincurran.org/com320/labs/netstat.html>
- <https://www.youtube.com/watch?v=UVdmVmJZuuY>
- <https://www.kaspersky.com/resource-center/definitions/what-is-steganography>
-

Glossary

Penetration tester - someone who simulates cyber attacks on a computing system to help test defences and identify weaknesses

Digital forensics - A sub section of forensic science that focuses specifically on analysing data from digital devices usually in regards to cyber crime

Bot - A bot is a software that runs automated tasks that typically mimic human actions. For example, messaging.

Cryptovirology - The study of encryption in malware

Packet - A small segment of data from a larger message that is sent over a network.

Plaintext - The unencrypted data that the user wishes to protect

Key-pair - The combination of the public and private key

Recursion - A programming technique that involves a subroutine calling itself for within creating a loop

Depth - first tree traversal algorithm - An algorithm for searching a tree. The algorithm starts at the root node and explores as far as possible along each branch before backtracking.

Breadth - first tree traversal algorithm - An algorithm for searching a tree you start at a source node and layer by layer through the graph, analysing the nodes directly related to the source node

Object Oriented Programming - A type of programming paradigm that organises software design around the data which is known as objects

DNS server - DNS servers translate domain names into IP addresses using lookup tables

Graph - A graph is a data structure composed of a collection of nodes and is used to represent the connections between those nodes. They are typically used to show the routes between nodes.

Full Code Appendix

GUI.py

```

Python
import tkinter as tk
from tkinter import *
from tkinter import ttk
from tkinter import messagebox
from yara_comparison import indepth_scan, quick_scan, compiled_rules,
connection
from quarantine_space import quarantine_module
from watch_directory import toggle_overwatch
from Virtual_Machine import virtualbox_testing
from process_analysis import process_analyse
import GUI_to_Yara
import configparser
from tkinter import PhotoImage
import webbrowser
import mysql.connector

config = configparser.ConfigParser()
config.read(r'C:\Users\rajan\Desktop\NEA - Copy\Scrubble 1.0\config.cfg')
print(config.read(r'C:\Users\rajan\Desktop\NEA - Copy\Scrubble
1.0\config.cfg'))

root = tk.Tk()

def sqlconnection(): # Setup to connect to db
    c = mysql.connector.connect(
        host=config["sql_settings"]["host"],
        username=config["sql_settings"]["username"],
        password=config["sql_settings"]["password"],
        database=config["sql_settings"]["database"]
    )

    if c.is_connected():
        print("Connected to SQL server successfully")

    return c

def main():
    root.geometry("800x500")
    root.title("Scrubble AV")
    root.configure(background="Lavender")
    img = PhotoImage(file=config["GUI_settings"]["Scrubble_Man"])
    root.iconphoto(False, img)

    def menu_setup():

```

```

menu = Menu(root)

file_menu = Menu(menu, tearoff=0)
file_menu.add_command(label='Close', command=root.quit)
menu.add_cascade(label='File', menu=file_menu)

help_menu = Menu(menu, tearoff=0)
help_menu.add_command(label='About', command=show_version)
menu.add_cascade(label='Help', menu=help_menu)

education_menu = Menu(menu, tearoff=0)
menu.add_cascade(label="Education Page", menu=education_menu)
education_menu.add_command(label="Education",
command=education_page)

user_menu = Menu(menu, tearoff=0)
user_menu.add_command(label="User 1", command=lambda:
select_user_number(1))
user_menu.add_command(label="User 2", command=lambda:
select_user_number(2))
user_menu.add_command(label="User 3", command=lambda:
select_user_number(3))
user_menu.add_command(label="User 4", command=lambda:
select_user_number(4))
user_menu.add_command(label="User 5", command=lambda:
select_user_number(5))
user_menu.add_command(label="Next User", command=lambda:
select_user_number())
menu.add_cascade(label="User Selection", menu=user_menu)

root.config(menu=menu)

def show_version():
    messagebox.showinfo("About", "Scrubble AV - Version 1.0\nAntivirus
built with Python")

def education_page():
    webbrowser.open("https://rajansumman.github.io/Scrubble/")

user_id = None

def select_user_number(user_number=None):

    current_user_index = 0

    def user_sql(user_number):
        try:

```

```

        cursor = connection.cursor()

        query = "SELECT user_id FROM users WHERE user_id = %s"
        cursor.execute(query, (user_number,))
        result = cursor.fetchone()

        if result:
            return result[0]
        else:
            return None
    except mysql.connector.Error as e:
        messagebox.showerror("Database Error", f"An error occurred:
{str(e)}")
        return None

    if user_number is None:
        user_number = current_user_index + 1

    user_id = user_sql(user_number)

    if user_id:
        GUI_to_Yara.set_user_id(user_id)
        messagebox.showinfo("User Selected", f"User {user_number}
selected (ID: {user_id})")
        current_user_index = user_number % 5 # Cycle back to 1 after
User 5
    else:
        messagebox.showerror("User Selection Error", f"User
{user_number} not found in the database.")

    menu_setup()

def quick_scan_button():
    quick_scanB = tk.Button(text="Quick Scan",
                            font=("Arial", 18),
                            bg="lavender",
                            command=lambda: quick_scan(compiled_rules,
connection, user_id))
    quick_scanB.pack(anchor="w", padx=20, pady=20)

def deep_scan_button():
    deep_scanB = tk.Button(text="Deep Scan",
                            font=("Arial", 18),
                            bg="lavender",
                            command=lambda: indepth_scan(compiled_rules,
connection, user_id))

```

```

        deep_scanB.pack(padx=20, pady=20, anchor="w")

def virtualbox_testing_button():
    vboxtestB = tk.Button(text="Test in Virtual Box",
                           font=("Arial", 18),
                           bg="lavender",
                           command=lambda: virtualbox_testing())
    vboxtestB.pack(padx=20, pady=20, anchor="w")

def delete_button():
    deleteB = tk.Button(text="Delete malware",
                         font=('Arial', 18),
                         bg="lavender",
                         command=lambda: quarantine_module())
    deleteB.pack(padx=20, pady=20, anchor="w")

def network_button():
    networkB = tk.Button(text='Start network analysis',
                         font=('Arial', 18),
                         bg='lavender',
                         command= lambda: process_analyse())
    networkB.pack(padx=20, pady=20, anchor='w')

def buttons():
    label = tk.Label(root, text="Scrubble AV!", font=("Arial", 18,
"bold"), anchor=tk.CENTER, bg="lavender")
    label.pack(padx=20)

    quick_scan_button()
    deep_scan_button()
    virtualbox_testing_button()
    delete_button()
    network_button()

    buttons()
    #toggle_overwatch()
    root.mainloop()

if __name__ == "__main__":
    main()
    connection.close()

```


Yara_Comparison.py

```

Python
import os
import time
import yara
import configparser
import mysql.connector
import datetime
import GUI_to_Yara

# Loads config file
config = configparser.ConfigParser()
config.read(r'C:\Users\rajan\Desktop\NEA - Copy\Scrubble 1.0\config.cfg')
print(config.read(r'C:\Users\rajan\Desktop\NEA - Copy\Scrubble
1.0\config.cfg'))

# Initialises malware list
malware = []
threat_ids = []

def get_dict_order():
    order_list = []
    cursor = connection.cursor()
    cursor.execute("SELECT rule_id FROM threats INNER JOIN yara ON
yara.rule_id = threats.yara_rule_id GROUP BY rule_id, rule_name ORDER BY
COUNT(*) DESC")
    result = cursor.fetchall()
    cursor.close()

    for (rule_id,) in result:
        order_list.append(rule_id)

    return order_list

def compile_yara_rules(yara_directory, connection):
    rules_dict = {}

    for root, _, files in os.walk(yara_directory):
        for file in files:
            if file.endswith(".yar") or file.endswith(".yara"):
                rule_path = os.path.join(root, file)
                filename = str(file)

```

```

        cursor = connection.cursor()
        query = "SELECT rule_id FROM yara WHERE rule_name=%s"

        try:
            cursor.execute(query, (filename,))
            result = cursor.fetchone()
            rule_id = result[0] if result else 0
            cursor.fetchall() # Clear any unread results

        except mysql.connector.Error as err:
            print(f"Error: {err}")

        finally:
            cursor.close()

        rules_dict[str(rule_id)] = rule_path

    print("Initial Rules Dictionary:", rules_dict)

    print("-----")

    order_list = get_dict_order()
    print("Order List after re-order:", order_list)

    print("-----")

    def order_dictionary(rules_dict, order_list):
        # Create ordered dictionary from the order list
        ordered_dict = {str(key): rules_dict[str(key)] for key in order_list
                        if str(key) in rules_dict}

        # Add remaining rules that are not in the order list
        for key, value in rules_dict.items():
            if key not in ordered_dict:
                ordered_dict[key] = value

        return ordered_dict

    # Merge and print the ordered rules
    rules_dict = order_dictionary(rules_dict, order_list)

    print("Final Ordered Rules Dictionary:", rules_dict)

    compiled_rules = yara.compile(filepaths=rules_dict)
    print("YARA rules compiled successfully.")

    return compiled_rules

```

```

def sqlconnection():
    c = mysql.connector.connect(
        host=config["sql_settings"]["host"],
        username=config["sql_settings"]["username"],
        password=config["sql_settings"]["password"],
        database=config["sql_settings"]["database"]
    )

    if c.is_connected():
        print("Connected to SQL server successfully")
    return c

def check_yara_rule(compiled_rules, file_path, file, yara_d, user_id):

    try:
        print(f"Comparing {file_path} to YARA rules")
        matches = compiled_rules.match(file_path)

        if matches:
            print(f"Matches found in {file_path}:")

            for match in matches:
                print(f"Rule name for match: {match.rule}")

                if match.rule == "HelloWorldRule":
                    print(f"Malware being appended is: {file_path} ")
                    malware.append(file_path)
                    time.sleep(3)
                    print("Malware successfully appended ", malware)

            if len(matches) >= 10:

                print(f"Malware being appended is: {file_path} ")
                malware.append(file_path)
                time.sleep(3)
                print("Malware successfully appended ", malware)

            else:
                print("While matches are present, there isnt enough of them
to warrant concern")

```

```

def find_yara_name(match, yara_d):
    for root, dirs, files in os.walk(yara_d):
        for file in files:
            if file.endswith(('yar', 'yara')):

                with open(os.path.join(root, file), 'r') as f:

                    if f"rule {match}" in f.read():

                        return file

yara_name = find_yara_name(match.rule, yara_d)

def get_rule_id(yara_name):
    cursor = connection.cursor()
    cursor.execute("SELECT rule_id FROM yara WHERE rule_name =
%s", (yara_name,))
    result = cursor.fetchone()[0]
    cursor.close()
    return result

rule_id = get_rule_id(yara_name)

def threat_sql(file_path, matches, file, rule_id, user_id):
    try:
        cursor = connection.cursor()
        cursor.execute("SELECT threat_id FROM threats ORDER BY
threat_id DESC LIMIT 1")
        previous_ThreatID = cursor.fetchone()[0]

        sql = "INSERT INTO threats (threat_id, file_path,
threat_name, severity_level, yara_rule_id, user_id) VALUES (%s, %s, %s, %s,
%s, %s)"

        val = (previous_ThreatID + 1, file_path, file,
len(matches), rule_id, user_id)

        cursor.execute(sql, val)
        connection.commit()
        print(cursor.rowcount, "record added")
        threat_ids.append(previous_ThreatID)

    except mysql.connector.Error as e:

```

```

        print(f"Error with DB: {e}")

        threat_sql(file_path, matches, file, rule_id, user_id)
    else:
        print(f"No match found in {file_path}.")
except Exception as e:
    print(f"Could not scan {file_path} due to error: {e}")

def bfs(root, files, compiled_rules, yara_d, user_id):
    for file in files:
        file_path = os.path.join(root, file)
        print(f"Scanning file: {file_path}")
        check_yara_rule(compiled_rules, file_path, file, yara_d, user_id)

def indepth_scan(compiled_rules, connection, user_id):
    user_id = GUI_to_Yara.get_user_id()
    if user_id is None:
        print("No user detected")
        return

    print(f"User: {user_id}")

    time.sleep(2)

    print("In-depth scan begun")
    start = time.time()

    def list_files_and_scan(starting_directory, compiled_rules):
        for root, dirs, files in os.walk(starting_directory):
            bfs(root, files, compiled_rules, yara_d, user_id)

    list_files_and_scan(starting_directory="C:\\",
compiled_rules=compiled_rules)
    end = time.time()

    time_taken = datetime.timedelta(seconds=end - start)

    print("Malware found: ")
    print(malware)

    for i in malware:
        try:

```

```

        j = str(i)
        if i == j:
            continue
        else:
            malware.remove(i)
    except:
        malware.remove(i)

print(malware)

total_threats_found = len(malware)
print(f"Threat ids : {threat_ids}")

print("Time taken:", time_taken)

try:
    cursor = connection.cursor()
    cursor.execute("SELECT scan_id FROM scan_logs WHERE scan_id=(SELECT
max(scan_id) FROM scan_logs)")
    previous_ID = cursor.fetchone()[0]

    sql = "INSERT INTO scan_logs (scan_id, scan_type, scan_time,
total_threats_found) VALUES(%s, %s, %s, %s)"
    val = (previous_ID + 1, "in_depth", str(time_taken),
total_threats_found)

    cursor.execute(sql, val)
    connection.commit()
    print(cursor.rowcount, "record inserted")
    cursor.close()
    connection.close()
except mysql.connector.Error as e:
    print(f"Error with Database: {e}")

def quick_scan(compiled_rules, connection, user_id):
    user_id = GUI_to_Yara.get_user_id()
    if user_id is None:
        print("No user detected")
        return

    print(f"User: {user_id}")
    time.sleep(2)

    print("Quick scan begun")
    start = time.time()

```

```

vulnerable_areas = ["C:/Users/rajan/Downloads"]

for area in vulnerable_areas:
    for root, dirs, files in os.walk(area):
        bfs(root, files, compiled_rules, yara_d, user_id)

end = time.time()
time_taken = datetime.timedelta(seconds=end - start)

print("Malware found: ")
print(malware)

for i in malware:
    try:
        j = str(i)
        if i == j:
            continue
        else:
            malware.remove(i)
    except:
        malware.remove(i)

print(malware)

total_threats_found = len(malware)
print(f"Threat ids : {threat_ids}")

print("Quick scan time taken:", time_taken)

try:
    cursor = connection.cursor()
    cursor.execute("SELECT scan_id FROM scan_logs WHERE scan_id=(SELECT
max(scan_id) FROM scan_logs)")
    previous_ID = cursor.fetchone()[0]

    sql = "INSERT INTO scan_logs (scan_id, scan_type, scan_time,
total_threats_found) VALUES(%s, %s, %s, %s)"
    val = (previous_ID + 1, "quick", str(time_taken),
total_threats_found)

    cursor.execute(sql, val)
    connection.commit()
    print(cursor.rowcount, "record inserted")
    cursor.close()

```

```

        connection.close()
    except mysql.connector.Error as e:
        print(f"Error with Database: {e}")

print("Sections in config:", config.sections())
yara_d = config["yara_comparison_settings"]["yara_directory"]

connection = sqlconnection()

compiled_rules = compile_yara_rules(yara_d, connection)

```

Index.html

```

JavaScript
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>education_page</title>
  </head>
  <body>
    <div id="heading_1">
      <h1><u>Staying Safe Online</u></h1>
    </div>

    <div id="heading_2">
      <h2><u>Why is online safety important</u></h2>
      <p>
        Between 2023 and 2024, Cyber attacks across the globe has seen an
        increase by around 30%. The losses made from cyber crime are
        expected to
        hit 24 trillion dollars by 2027, if cyber crime continues to

```


exponentially increase at the rate it currently is. It can be extremely harmful

</p>

</div>

<div id="heading_3">

<h3><u>Protection of Personal information</u></h3>

<p>

Additionally, because we live more of our lives through our screens the more personal details and information we keep on the digital landscape. This places us into an increasing level of risk when this information ends up in the wrong hands. For example information such as: personal communications, financial transactions, health records and past social media posts are all components of what is known as your digital footprint. All of this information can be traced back to you and can be utilised in many ways, sometimes maliciously.

</p>

</div>

<div id="heading_3">

<h3><u>Protection of children</u></h3>

<p>

Children are the most vulnerable group of internet users due to them being unable to recognise potential risks. This makes them targets for cybercriminals and online threats. Around 25% of 5-7 year olds have a smart phone, and 76% of them will use a tablet. 32% of parents have also stated that they would let their child use social media entirely on their own. These statistics are taken from studies conducted between 2017 to present day.

Furthermore, with the rise of social media, there has been an increase in cyberbullying and online harassment. 73% of young women have received unwanted sexual content online. 26.5% of American teenagers have experienced cyberbullying within the last 30 days. These statistics clearly indicate that online crime is a hugely widespread problem and steps should be taken to try and protect people online.

</p>

</div>

```

<div id="heading_3">
  <h3><u>Social Engineering</u></h3>
  <p>
    Social engineering in the context of cyber security is the process
of
    manipulating and deceiving someone into giving sensitive
information or
    control over a computer system. Social engineering is done entirely
in
    person and therefore relies on human interaction and psychology to
work,
    it cannot be defended against using software. Therefore it is vital
people can understand what social engineering is and how harmful it
can
    be.
    <br />
    <br />
    98% of cyber attacks will involve some sort of social engineering 75%
of
    companies have reported being a victim of a socially engineered
attack
    in the year 2020.
  </p>
</div>

<div id="heading_2">
  <h2><u>Practices for staying safe online</u></h2>
</div>

<div id="heading_3">
  <h3><u>Securing Connections</u></h3>
  <p>
    1. Only connect to internet connections you know are secure.
Especially
    when handling sensitive information. You can tell if a webpage is
secure
    if it has HTTPS in the URL.
    <br />
    Some good practices to use when connecting to networks you are not
familiar with is using a VPN (virtual private network) for added
security via encryption.
    <br />
    <br />
    2. Disabling bluetooth on devices with very private information can
prevent hacking. However this is most likely overkill however it is
something to keep in mind
  </p>

```

```

</div>

<div id="heading_3">
  <h3><u>Protecting Devices and Accounts</u></h3>
  <p></p>
  <h4>Secure passwords</h4>
  1. One of the best ways to ensure that your devices and accounts are
  secure from outside attack is to use secure unique passwords for
  everything. If your the forgetful type, a password manager can be
  extremely useful for keeping track and making sure you dont forget. A
  great resource to check if your password is secure is to use
  <a href="https://www.passwordmonster.com/"
  >https://www.passwordmonster.com/</a>
  >
  <br />
  <h4>Keeping software updated</h4>
  2. Additionally, all software should be updated to the latest possible
  version. This is because software engineer will include security
  enhancements and patches to bugs that hackers may attempt to exploit.
  This
  greatly reduces the chance that your device or account will be a
  target.
  <br />
  <br />
  <h4>Encryption</h4>
  3. Using only fully encrypted communication channels, especially when
  exchanging private or sensitive information is extremely important.
  Encryption is the process of protecting data by scrambling and
  altering
  that data into a format that is unreadable by humans. This is done
  using
  whats known as keys. Keys are essentially the algorithm that is
  resonsible
  for scrambling the data, therefore the data can only be read by
  individual
  who possess the encryption key. This means that outside hackers who
  may
  attempt to intercept your message that you are sending cannot
  physically
  read it as its simply a bunch of jumbled up nonsense. For this reason
  I
  highly suggest double checking messaging services or websites where
  sensitive data is being exchanged to make sure that it is encrypted.
  You
  can again check this by ensuring that the URL contains HTTPS.
  </div>
</body>
<style>

```

```
body {  
  background-color: #d4c1ff;  
  font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI",  
    Roboto, Oxygen, Ubuntu, Cantarell, "Open Sans", "Helvetica Neue",  
    sans-serif;  
}  
  
#heading_1 {  
  text-align: center;  
}  
  
#heading_2 {  
  padding: 10px;  
}  
  
#heading_3 {  
  padding: 30px;  
}  
</style>  
</html>
```

Config.cfg

```
Unset  
[yara_comparison_settings]  
vulnerable_areas = C:\Users\rajan\Downloads  
yara_directory = C:\Users\rajan\Desktop\NEA\Scrubble 1.0\MyRules  
  
[sql_settings]  
host=localhost  
username=root  
password=Monopoly2020!  
database=test  
  
[quarantine_space_settings]  
quarantine_folder_path = C:\Users\rajan\Desktop\Quarantine Path  
  
[watch_directory_settings]
```

```
path_to_watch = C:\\  
  
[GUI_settings]  
Scrubble_Man = C:\\Users\\rajan\\Desktop\\NEA\\Macot Pictures\\Mascot-DarkDude.png  
  
[virtual_machine_settings]  
vm_name = Windows 11  
guest_username = rajansumman  
guest_password = Monopoly2020!
```